

A Review on Edge Large Language Models: Design, Execution, and Applications

YUE ZHENG, Zhejiang University of Technology, Hangzhou, China

YUHAO CHEN, Zhejiang University, Hangzhou, China

BIN QIAN, Zhejiang University, Hangzhou, China

XIUFANG SHI, Zhejiang University of Technology, Hangzhou, China

YUANCHAO SHU, Zhejiang University, Hangzhou, China

JIMING CHEN, Zhejiang University, Hangzhou, China

Large language models (LLMs) have revolutionized natural language processing with their exceptional understanding, synthesizing, and reasoning capabilities. However, deploying LLMs on resource-constrained edge devices presents significant challenges due to computational limitations, memory constraints, and edge hardware heterogeneity. This survey provides a comprehensive overview of recent advancements in edge LLMs, covering the entire lifecycle—from resource-efficient model design and pre-deployment strategies to run-time inference optimizations. It also explores on-device applications across various domains. By synthesizing state-of-the-art techniques and identifying future research directions, this survey bridges the gap between the immense potential of LLMs and the constraints of edge computing.

CCS Concepts: • **Computing methodologies** → **Artificial intelligence**; • **Computer systems organization** → **Embedded and cyber-physical systems**; • **Human-centered computing** → **Ubiquitous and mobile computing**;

Additional Key Words and Phrases: Edge computing, large language models, resource-efficient optimizations, on-device inference, LLM applications

ACM Reference Format:

Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu, and Jiming Chen. 2025. A Review on Edge Large Language Models: Design, Execution, and Applications. *ACM Comput. Surv.* 57, 8, Article 209 (March 2025), 35 pages. <https://doi.org/10.1145/3719664>

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 92467301, U23A20326, 62293511, and 62372414, the Key Research and Development Program of Zhejiang Province under Grant 2025C01061 and 2025C01012, and in part by the ZJUCSE-Enflame cloud and edge intelligence joint laboratory.

Authors' Contact Information: Yue Zheng, Zhejiang University of Technology, Hangzhou, China; e-mail: zhengyue.peace@gmail.com; Yuhao Chen, Zhejiang University, Hangzhou, China; e-mail: csechenyh@zju.edu.cn; Bin Qian, Zhejiang University, Hangzhou, China; e-mail: bin.qian0718@gmail.com; Xiufang Shi, Zhejiang University of Technology, Hangzhou, China; e-mail: xiufangshi@zjut.edu.cn; Yuanchao Shu (Corresponding author), Zhejiang University, Hangzhou, China; e-mail: yeshu@zju.edu.cn; Jiming Chen (Corresponding author), Zhejiang University, Hangzhou, China; e-mail: cjm@zju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 0360-0300/2025/03-ART209

<https://doi.org/10.1145/3719664>

1 Introduction

Transformer-based **large language models (LLMs)** have made significant strides in recent years, reshaping the landscape of **natural language processing (NLP)**. This rapid evolution has led to the emergence of several open-source LLMs, including Meta's LLaMA family [38, 176, 177], Google's Gemma [169, 170], and more recently, DeepSeek AI's DeepSeek series [58, 113]. The success of LLMs stems from their exceptional capabilities in natural language understanding, synthesizing, reasoning, and generation [35, 36], driving breakthroughs in applications like document summarization, question answering, and text reformulating [92, 97, 115]. These advancements have had profound implications across both academic and industrial domains, enabling the development of widely adopted tools like ChatGPT [135], Copilot [129], and Gemini [168]. The continued progress in LLMs underscores their transformative impact on artificial intelligence [23, 84, 194], human-computer interaction [61, 72, 87], and beyond.

While cloud-based deployment has traditionally supported LLMs' computational demands, there is a growing need to bring these models to resource-constrained edge devices, including personal agents [147, 194], office assistants [61, 168], and industrial **Internet of Things (IoT)** systems [76, 174]. Edge-based LLMs—executed directly on devices—offer key advantages: Firstly, local inference ensures faster responses and functionality without internet connectivity [19], which is critical for applications in robotics and autonomous systems [23, 31, 198]. Secondly, processing sensitive data on-device eliminates risks associated with cloud transmission [42, 168]. Lastly, on-device learning enables models to adapt to user-specific preferences and contexts [13, 86, 137, 143].

However, deploying LLMs on resource-constrained edge devices presents significant challenges. Firstly, **the computational and memory constraints** impose substantial limitations on LLM loading and inference. LLMs often consist of billions of parameters, resulting in massive memory footprints that exceed the RAM capacities of most edge devices [27]. For example, an LLaMA-2 [177] model with 7B parameters requires over 8 GB of memory even in FP16 precision. Without compression techniques, edge devices risk latency spikes and memory overflow during model loading [112]. Moreover, the quadratic complexity of the self-attention mechanism with respect to sequence length exacerbates computational demands, creating severe throughput bottlenecks on edge **Central Processing Units (CPUs)**, **Graphics Processing Units (GPUs)**, or **Neural Processing Units (NPUs)** [156].

Secondly, **the heterogeneous nature of edge computing devices** complicates runtime inference optimizations. Edge devices range from smartphones with ARM CPUs and limited memory to IoT devices equipped with low-power chips. On mobile devices, frameworks like llama.cpp [49] and MLC LLM [171] optimize computational operators, while edge GPUs adopt approaches like vLLM [95] to alleviate memory bandwidth limitations and enhance throughput. Effective hardware-software co-design is critical to align workloads with hardware-specific capabilities. Additionally, the choice of hardware (e.g., CPUs, GPUs, or NPUs) and its integration with software frameworks directly affect inference efficiency, necessitating adaptable solutions tailored to diverse edge environments [225].

Lastly, **developing practical edge applications** remains challenging, particularly in bridging centralized LLM processing with distributed edge scenarios. In personal and enterprise applications, frameworks such as AutoDroid [194] and WebAgent [61] demonstrate the complexities of maintaining responsiveness and accuracy for task automation. For industrial systems like autonomous vehicles [23, 174], precise task prioritization and dynamic resource allocation are essential to balance LLM inference with real-time control processes. These domain-specific optimizations are vital to ensure LLMs meet real-world latency and reliability requirements on resource-constrained devices.

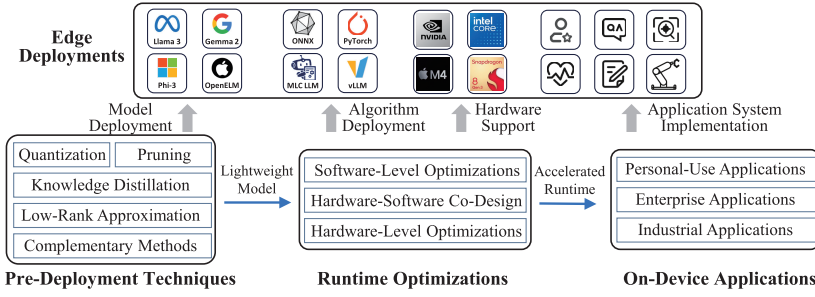


Fig. 1. Edge LLMs deployment, optimization, and application pipeline.

To address these challenges, we have devised a comprehensive optimization pipeline that integrates techniques across the entire lifecycle of edge-based LLM deployment, as illustrated in Figure 1. Starting with pre-deployment methods such as quantization, pruning, and knowledge distillation, the pipeline enables the creation of compact, resource-efficient models that preserve performance while reducing computational demands. These models are then deployed on edge devices, where runtime optimizations—spanning software-level strategies, hardware-level enhancements, and hardware-software co-design—ensure seamless adaptation to heterogeneous environments. Finally, the optimized models power a variety of on-device applications, from personal assistants to enterprise systems and industrial solutions, showcasing the practical impact of edge LLMs. This unified process effectively addresses key deployment challenges, demonstrating how offline compression and real-time optimizations together enable diverse real-world applications.

Following this pipeline, this survey aims to provide a comprehensive exploration of the key areas involved in enabling LLMs on edge and mobile devices, including three crucial aspects, as summarized in Figure 2. Specifically,

- **Offline Pre-Deployment Model Design Techniques.** It focuses on compression models to reduce size and ease deployment on edge devices. Traditional methods like quantization, pruning, knowledge distillation, and low-rank approximation face unique challenges with LLMs due to their scale, Transformer architecture, and diverse tasks [2, 77]. These challenges have motivated novel compression methods tailored for LLMs. Quantization reduces LLM size by representing weights and activations with fewer bits [112, 153]. Pruning removes unnecessary attention heads or other Transformer components, either structurally or unstructurally [94, 197]. Knowledge distillation transfers knowledge to smaller models [82, 110]. Low-rank approximation exploits matrix redundancy for efficient compression [69, 109]. Complementary methods, such as advanced pre-training strategies, data curation, and architectural optimization, further enhance compression effectiveness [127, 130, 169].
- **Online Runtime Inference Optimizations.** It introduces inference optimization techniques that improves the LLM performance on resource-constrained edge devices. Key strategies include software-level optimizations, hardware-software co-design, and hardware-level optimizations. Software-level optimizations encompass resource-aware scheduling strategies for cloud-edge collaboration [17, 161, 227], single-device inference scenarios [51, 154, 216], and lightweight frameworks for efficient memory management and tensor operations [95, 157]. Hardware-software co-design integrates software algorithms with specific hardware capabilities, facilitating efficient hardware profiling and enabling the implementation of hardware-aware inference algorithms [56, 186]. Hardware-level optimizations introduce commonly used edge hardware devices, highlighting their innovations in on-device LLM inference [172, 213].

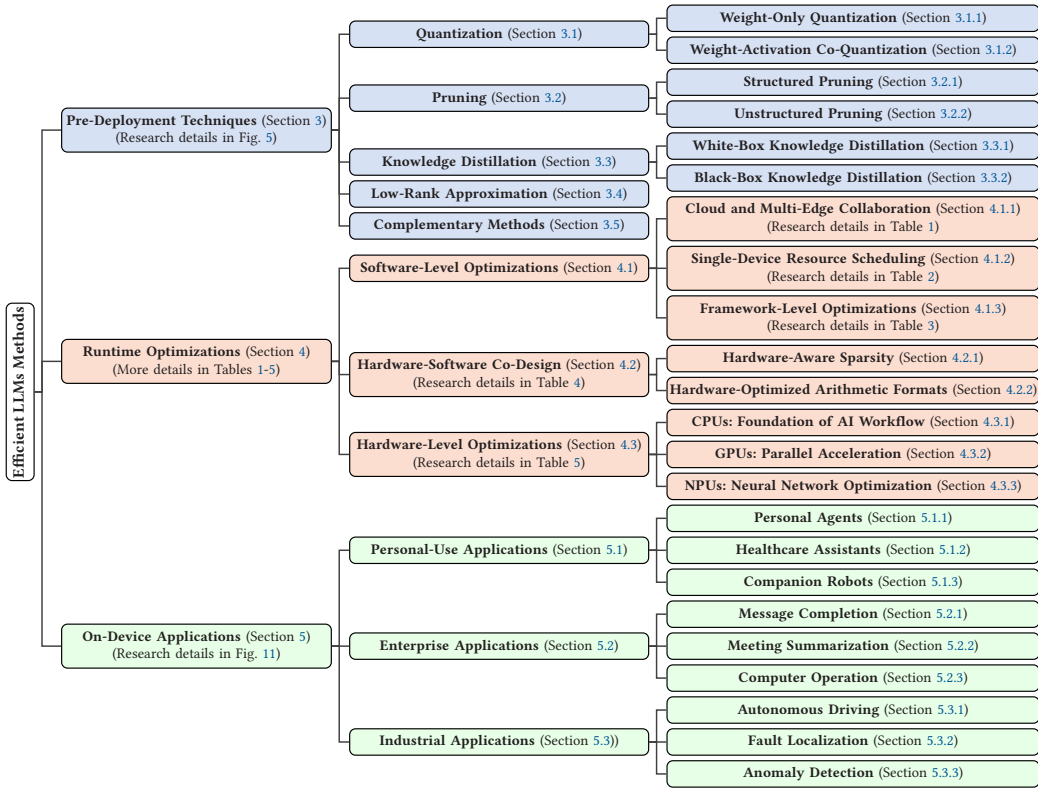


Fig. 2. A comprehensive overview of on-device LLMs, with detailed research categorized into: Pre-Deployment Techniques (Section 3, research details in Fig. 5), Runtime Optimizations (Section 4, details distributed across Tables 1-5), and On-Device Applications (Section 5, research details in Fig. 11). Each branch represents a key research direction with detailed methodology and implementation discussions in corresponding sections.

— **On-Device LLM-Based Applications.** It showcases the practical impact of on-device LLMs across personal, enterprise, and industrial domains. In personal applications, they power AI assistants for tasks such as daily management, healthcare monitoring, and companionship, offering privacy-preserving and low-latency interactions [72, 119, 194]. In enterprise settings, on-device LLMs enhance productivity through message completion, meeting summarization, and secure local processing of sensitive data [105, 175, 234]. In industrial scenarios, they enable real-time and local processing capabilities like autonomous driving, fault localization, and anomaly detection, improving efficiency and safety in complex environments [54, 84, 174].

By employing these innovative techniques and methodologies, developers can harness the benefits of reduced model sizes and improved computational efficiency, facilitating the seamless integration of LLMs on edge devices. This advancement not only improves edge computing performance but also broadens the applicability of LLMs in resource-constrained environments, potentially revolutionizing the landscape of edge AI applications.

The remainder of this article is structured as follows: Section 2 examines the widening gap between LLM complexity and edge device capabilities, reviews related work on efficient LLMs and edge computing, and analyzes research trends in on-device LLM optimizations, establishing the

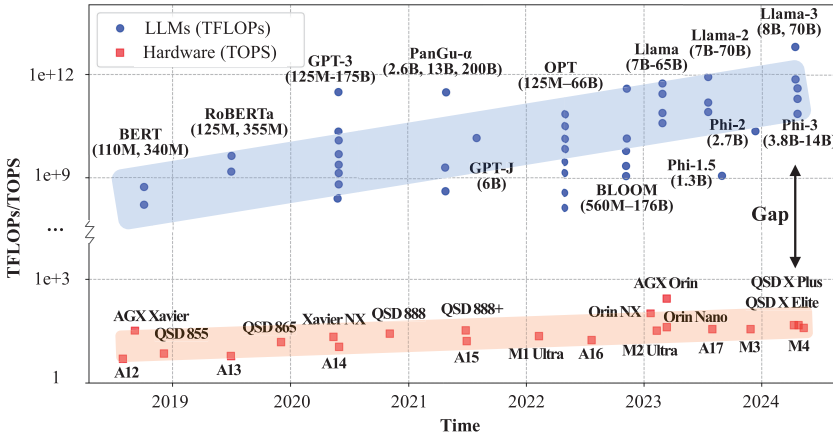


Fig. 3. LLMs (TFLOPs) vs. Edge Devices (TOPS) Over Time. (QSD: Qualcomm Snapdragon.)

context for our survey. Sections 3 and 4 provide a comprehensive examination of the state-of-the-art approaches for offline pre-deployment techniques and online runtime optimizations, respectively. Section 5 delves into the on-device applications of LLMs, highlighting their vast potential. Section 6 discusses future directions and open challenges in the field of on-device LLMs, while Section 7 concludes the survey, summarizing the key takeaways and insights gained.

2 Background and Related Work

The rapid advancement of LLMs and the increasing demand for edge computing have led to a growing interest in deploying these powerful AI models on resource-constrained devices [112]. However, this endeavor is hindered by a significant disparity between the computational complexity of LLMs and the capabilities of edge devices. Figure 3 illustrates this widening gap, showing the evolution of estimated LLM pre-training FLOPs [18, 148] (measured in TFLOPs) compared to the AI performance of edge devices [30, 34, 145] (measured in TOPS) over time. The pre-training FLOPs for LLMs are estimated using the widely accepted heuristic $C \approx 6ND$, where N denotes the model's parameter count and D represents the total number of tokens used during pre-training [85].

This graph clearly illustrates the widening gap between the rapidly increasing computational complexity of LLMs and the relatively slow improvement in edge device capabilities. While LLMs have experienced a steep rise in estimated pre-training FLOPs, the AI performance of edge devices has improved at a much slower pace. This growing disparity underscores the critical need for researching efficient LLM deployment and implementation methods, which is precisely the focus of our survey.

While prior surveys in edge computing research have been conducted on efficient learning techniques on **Deep Neural Network (DNN)** architectures, such as **Convolutional Neural Networks (CNNs)** and **Recurrent Neural Networks (RNNs)** [98, 190], they have not adequately addressed the unique challenges posed by LLMs, including their larger model sizes and the complexities introduced by attention mechanisms. While research has addressed resource management [222] and security enhancements [192] in edge environments, these studies primarily focus on general deep learning, neglecting the specific needs of transformer-based LLMs in mobile edge computing.

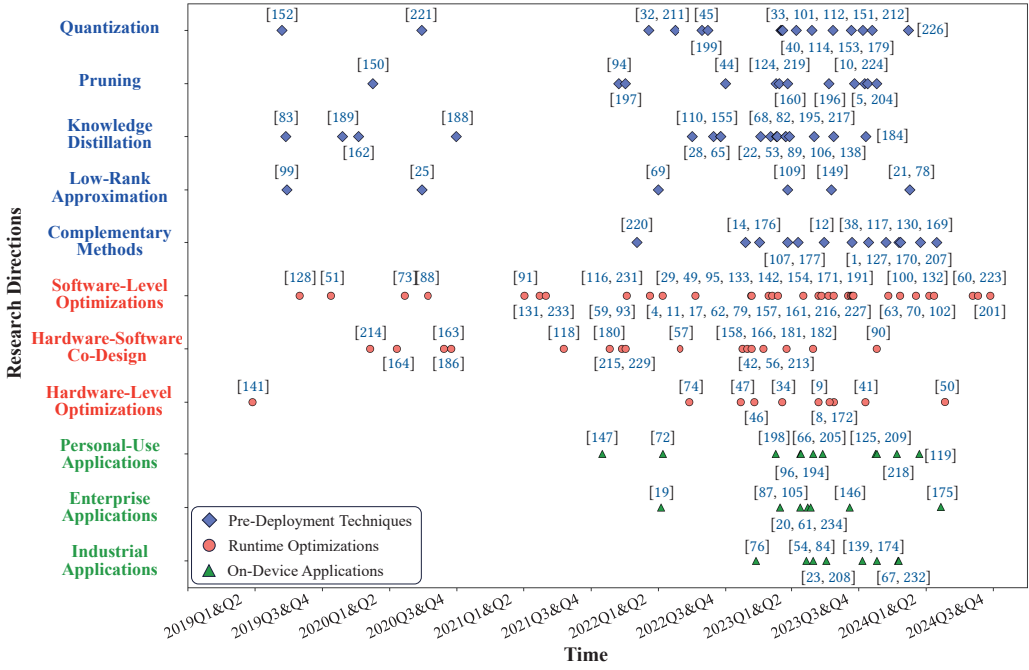


Fig. 4. Temporal Distribution of On-Device LLM Research.

Complementing these efforts, research in the realm of NLP has also made significant strides. Xu and McAuley [200] review methods for enhancing the efficiency of model compression and acceleration for pre-trained LLMs. Hedderich et al. [64] survey approaches for improving performance in low-resource NLP settings. Wan et al. [185] provide a comprehensive review of efficient LLMs research, organizing the literature into model-centric, data-centric, and framework-centric approaches. Treviso et al. [178] synthesize methods for conducting NLP under constraints of limited data, time, storage, or energy, emphasizing the tradeoffs between performance and resource consumption. However, these works do not specifically address the challenges of deploying LLMs in edge environments. As a result, there is a critical need for focused research in this area.

Our survey uniquely provides a comprehensive analysis of LLMs in edge environments. The two most relevant surveys are Mobile Edge Intelligence for LLMs [144], which focuses primarily on collaborative resource management across different computing nodes, and Personal LLM Agents [108], which explores the applications and scenarios of LLM assistants. However, the former does not address framework- and hardware-level optimizations for edge devices, and the latter lacks a systematic analysis of runtime optimizations on edge devices. To bridge this gap, we offer a holistic, top-down perspective on LLMs for edge devices, encompassing the entire optimization pipeline from offline pre-deployment model design techniques to online runtime inference optimizations and on-device LLM-based applications across various sectors. Our analysis covers model architectures, compression strategies, software-level optimizations, hardware-software co-design, and hardware enhancements for Transformer-based architectures at the edge. Additionally, we examine on-device application systems designed to maximize LLM performance under resource constraints. This multi-faceted approach distinguishes our survey, addressing challenges and solutions across the entire optimization pipeline for edge-deployed LLMs.

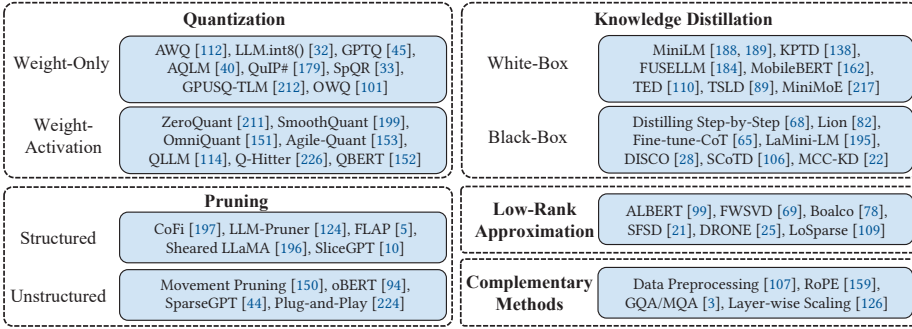


Fig. 5. An Overview of Offline Pre-deployment Model Design Techniques and Literature.

Figure 4 shows the evolution of on-device LLM research from 2019 to 2024, categorized into pre-deployment techniques (blue), runtime optimizations (purple), and on-device applications (green). Offline pre-deployment techniques, such as quantization, pruning, knowledge distillation, and low-rank approximation have been consistently researched throughout the period. Online runtime optimizations, including software-level optimizations, hardware-software co-design, and hardware-level optimizations, have gained traction from 2022. The emergence of on-device applications for personal, enterprise, and industrial use cases is particularly notable in the latter half of the timeline, indicating a growing trend towards edge AI and mobile LLM deployments. This highlights the rapid advancement and diversification of LLM deployment for resource-constrained environments, emphasizing the growing importance of efficient on-device AI. Our survey provides a comprehensive analysis of these trends, offering a foundation for future research and practice in this field.

3 Offline Pre-Deployment Model Design Techniques

The proliferation of LLMs has engendered a burgeoning demand for their deployment on mobile and edge devices, driven by the imperative for enhanced privacy, reduced latency, and improved service availability in connectivity-constrained environments. This paradigm shift towards edge computing for LLMs. However, it presents significant challenges due to the LLMs' inherent computational complexity and substantial memory requirements [77]. Consequently, offline pre-deployment model design techniques have emerged as a pivotal strategy, aiming to substantially reduce the computational and memory footprint of LLMs while preserving their performance integrity. These techniques, applied prior to the models' deployment on target edge devices, facilitate efficient execution in resource-constrained environments.

As illustrated in Figure 5, these pre-deployment techniques encompass five primary categories: quantization, pruning, knowledge distillation, low-rank approximation, and complementary methods.

3.1 Quantization

Quantization is a compression technique that reduces the precision of numerical values in models, providing significant deployment advantages for edge devices. However, applying conventional methods to LLMs is challenging due to the architectural complexity of Transformer-based models, which rely heavily on attention mechanisms and high-dimensional representations [77]. These characteristics result in precision-sensitive tasks, and the high dynamic activation ranges in these models exacerbate quantization difficulties, often leading to performance degradation [15]. To ad-

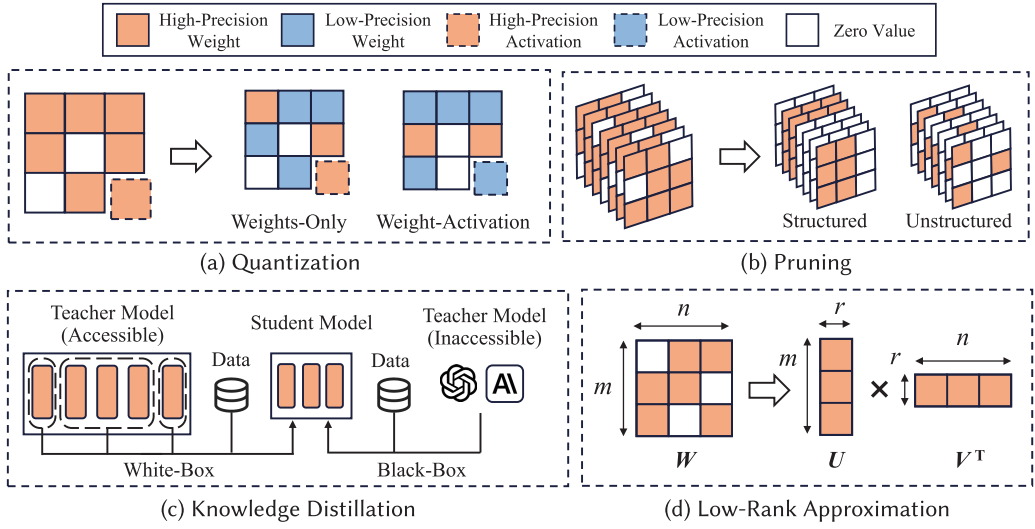


Fig. 6. Comparative schematics of offline compression methods for LLMs.

dress these challenges, specialized quantization methods have been developed, typically focused on two main areas:

- Weight quantization: Reduces the precision of model weights.
- Activation quantization: Reduces the precision of intermediate activations.

As shown in Figure 6(a), it can be categorized into weight-only quantization and weight-activation co-quantization.

3.1.1 Weight-Only Quantization. Weight-only quantization reduces the precision of model weights from high-precision data types (e.g., 32-bit floating point) to lower-precision ones (e.g., 8-bit integers). This reduces memory usage and can speed up inference on resource-constrained devices. For instance, Dettmers et al. [32] propose LLM.int8(), which reduces memory requirements during inference without sacrificing performance. Similarly, Frantar et al. [45] introduce GPTQ, a post-training method that compresses LLM weights to lower bits, addressing layer-wise quantization challenges with the OBQ method [43]. Moreover, Lin et al. [112] propose AWQ, which preserves salient weights at high precision while quantizing others, optimizing computational complexity and energy consumption. Other weight-only quantization methods, such as AQLM [40], QuIP# [179], and GPUSQ-TLM [212] also contribute to the advancements in this field by exploring different quantization strategies and optimization techniques.

Despite these advances, managing quantization errors in outlier weights remains a challenge. Techniques such as SpQR [33] and OWQ [101] address this issue by storing outlier weights at higher precision, improving compression efficiency without compromising performance.

3.1.2 Weight-Activation Co-Quantization. While weight-only quantization offers benefits, it may leave activation values uncompressed. Weight-activation co-quantization, which quantifies both weights and activations, provides further compression. For example, ZeroQuant [211] combines group-wise weight and token-wise activation quantization. SmoothQuant [199] offering lossless 8-bit quantization by using per-channel scaling transformations. Furthermore, Agile-Quant [153] and Q-Hitter [226] employ activation-aware techniques to balance performance and real-time inference speed. In addition to the advancements discussed, other established technologies,

including QBERT [152] and TernaryBERT [221] have also played a role in the development of weight-activation co-quantization for edge environments.

Despite these advancements, handling outlier issues in co-quantization remains a challenge. Methods like QLLM [114] and OmniQuant [151] address activation and weight outliers by leveraging adaptive calibration and learnable transformations, respectively.

In summary, quantization techniques for LLMs strike a balance between model compression, performance, and computational complexity. Weight-only methods, such as LLM.int8() [32], are ideal for quick deployment with moderate compression, while weight-activation co-quantization approaches like ZeroQuant [211] offer higher compression at the cost of increased complexity and potential accuracy loss.

3.2 Pruning

Pruning is a key technique for optimizing LLMs by reducing the number of parameters, leading to smaller model sizes and faster inference. However, pruning in LLMs is challenging due to the complexity of their architecture and the varying significance of components such as attention heads. Conventional pruning methods, effective in CNNs, face limitations when applied to LLMs [104]. As shown in Figure 6(b), specialized pruning techniques for LLMs are generally categorized into structured and unstructured pruning, each with distinct tradeoffs.

3.2.1 Structured Pruning. Structured pruning reduces the size of neural networks by removing entire structural components, such as neurons, channels, or layers. For example, CoFi [197] uses multiple pruning masks at different granularities to simultaneously remove layers and attention heads. LLM-Pruner [124] employs gradient-based pruning, removing non-critical units while preserving model performance, and incorporates LoRA [71] to recover performance post-pruning. LoRAPrune [219] leverages LoRA's weights and gradients for importance estimation. Sheared LLaMA [196] prunes specific layers and dimensions, using dynamic batch loading for domain-specific loss metrics, ideal for resource-constrained edge devices. SliceGPT [10] projects transformer block signal matrices onto principal components, removing redundant columns or rows to reduce size. FLAP [5] formulates importance metrics, enabling adaptive search for the optimal compressed model and implementing compensation mechanisms to mitigate performance loss.

3.2.2 Unstructured Pruning. Unstructured pruning removes individual weights or neurons, resulting in sparse models that are harder to optimize. Movement Pruning [150] adapts pruning decisions based on weight dynamics during fine-tuning, preserving important weights that exhibit significant movement. oBERT [94] introduces a second-order pruning method supporting both unstructured and block pruning. SparseGPT [44] treats pruning as a sparse regression problem, allowing for one-shot pruning without retraining. Plug-and-Play [224] integrates activation-based importance to prune weights selectively, further improving pruning robustness in large-scale models. Wanda [160] prunes weights with the smallest magnitudes multiplied by the corresponding input activations, on a per-output basis. BESA [204] targets the overall pruning error with respect to individual transformer blocks and allocates layer-specific sparsity in a differentiable manner.

Overall, pruning techniques for LLMs offer various strategies for balancing size reduction and performance retention. Structured pruning methods like CoFi [197] and LLM-Pruner [124] provide controlled reductions, preserving architecture integrity, while unstructured methods such as Movement Pruning [150] and oBERT [94] offer greater flexibility but may result in irregular, sparse models. In practice, the choice of method depends on the specific deployment scenario, considering the tradeoffs between model size, computational efficiency, and task performance.

3.3 Knowledge Distillation

Knowledge distillation transfers knowledge from complex teacher models to simpler student models to create computationally efficient alternatives without sacrificing performance. This process reduces model size, computational costs, and deployment requirements while enhancing the student model's diversity and stability. However, distilling knowledge from large teacher models, such as LLMs, remains challenging due to the difficulty in transferring internal representations and the complexity of attention mechanisms in Transformers [77]. As illustrated in Figure 6(c), distillation methods for LLMs are categorized into white-box and black-box approaches, each tailored to handle the scale and intricacies of LLMs.

3.3.1 White-Box Knowledge Distillation. White-box knowledge distillation leverages access to the teacher model's architecture and parameters, using internal features and logits for knowledge transfer. MiniLM [189] extracts knowledge from the final Transformer [183] layer, alleviating the complexity of layer-to-layer mapping, while MiniLMv2 [188] extends this for task-agnostic compression. Other significant contributions include MobileBERT [162], and TinyBERT [83] collectively laying the foundation in this area.

To mitigate capacity mismatches between teacher and student models, techniques such as reverse Kullback–Leibler Divergence in MiniLLM [53] and layer-wise alignment in TED [110] improve the effectiveness of distillation. KPTD [138] and TSLD [89] further refine the process through entity-based transfer and token-scaled logit distillation, respectively. MiniMoE [217] addresses the capacity gap by employing a **Mixture-of-Experts (MoE)** model, advancing the scalability of white-box distillation for LLMs. FUSELLM [184] integrates the capabilities of existing LLMs and transfers them into a single one, thereby elevating the capabilities of the target model. These advancements contribute to the evolving landscape of white-box knowledge distillation.

3.3.2 Black-Box Knowledge Distillation. Black-box knowledge distillation focuses solely on the outputs of the teacher model, bypassing internal model details. This approach is valuable when teacher models are proprietary or deployed via APIs. **Chain-of-Thought (CoT)** distillation aims to transfer reasoning abilities from LLMs to smaller student models [68, 106]. For example, Distilling Step-by-Step [68] uses LLM rationales to supervise task-specific models, improving dataset quality and model performance. Fine-tune-CoT [65] successfully transfers reasoning from large models (over 100B parameters) to students with as few as 0.3B. SCoTD [106] further enhances performance in both supervised and few-shot tasks. Other methods for CoT distillation include Socratic CoT [155] and MCC-KD [22].

Black-box distillation also includes methods that focus on instruction-following, a crucial ability for LLMs in real-world applications. Lion [82] employs adversarial distillation to generate complex instructions, producing a 13B-parameter model comparable to ChatGPT [135]. DISCO [28] distills counterfactual data to enhance robustness, while LaMini-LM [195] uses diverse instruction sets to compress large models effectively.

In conclusion, white-box methods, such as MiniLM [189], excel in scenarios where model internals are accessible. Conversely, black-box methods, such as Lion [82], are valuable in industrial or proprietary contexts where access to model internals is not accessible.

3.4 Low-Rank Approximation

Matrix factorization techniques, such as principal component analysis and regularized matrix factorization, have been pivotal in improving the generalization and interoperability of CNNs and RNNs. These methods reduce high-dimensional data to lower-dimensional spaces, enhancing model performance [77]. However, the large-scale parameters of Transformer-based LLMs pose challenges to traditional factorization approaches, as their computational complexity and unique

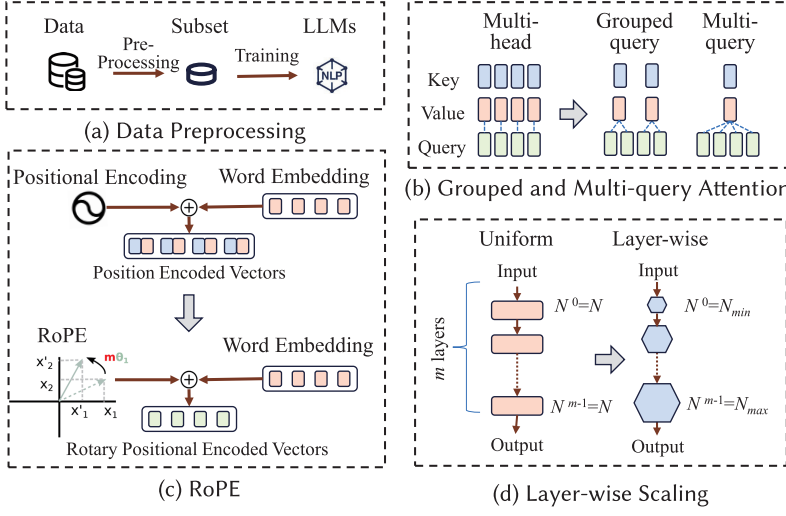


Fig. 7. Illustrations of complementary methods for designing efficient LLMs.

structural elements, such as multi-head attention and feed-forward networks, require specialized adaptations [48]. To address these, low-rank approximation has emerged as a promising strategy within the Transformer framework [21, 25].

As illustrated in Figure 6(d), this technique approximates a high-dimensional matrix $W_{m \times n}$ with the product of two lower-rank matrices, $U_{m \times r}$ and $(V^T)_{r \times n}$, where r is much smaller than m and n . For example, ALBERT [99] applied low-rank approximation to vocabulary embeddings, decoupling hidden layer size from vocabulary size. FWSVD [69] enhances singular value decomposition by incorporating Fisher information to weight parameter importance, while DRONE [25] optimizes weight matrix compression by leveraging data distribution. Innovations such as LoSparse [109] introduce a method for separating coherent and incoherent neuron components, outperforming traditional pruning approaches. Additionally, inherent low-rank characteristics in LLMs have led to Bayesian optimization-based feature compression techniques [78]. SFSD [21] refine feature space approximation, and fast randomized algorithms using Gaussian sketches facilitate efficient low-rank factorization on consumer-grade hardware [149].

In summary, low-rank approximation effectively reduces parameters in LLMs, making it ideal for optimizing large-scale matrices like embedding layers and attention weights [206]. By minimizing redundancy, it significantly reduces storage and computation costs while maintaining performance.

3.5 Complementary Methods

Recent research in the field of LLMs has witnessed a paradigm shift towards developing innovative approaches specifically tailored for compact models, typically with around 10B parameters or less. As illustrated in Figure 7, these technological advancements include:

- Data Preprocessing: Meticulous curation of high-quality training data [55].
- Grouped query and Multi-query Attention: Optimization of attention mechanisms [3].
- **Rotary Positional Embedding (RoPE)**: Advanced positional information encoding [159].
- Layer-wise Scaling: Strategic distribution of parameters across model layers [126].

The integration of these techniques has resulted in compact LLMs to excel on resource-constrained edge devices, supporting high-performance AI applications in diverse scenarios. For

instance, Meta's LLaMA series [6, 38, 176, 177] have achieved remarkable performance gains through efficient pre-training on carefully curated datasets and leverages grouped query attention and RoPE to improve inference speed, reduce cache size, and expand context lengths. Similarly, Microsoft's Phi series [1, 55, 130] showcases the impact of data preprocessing, using high-quality synthetic and filtered datasets to outperform larger models across benchmarks. Google's Gemma models [169, 170] combine multi-query attention and RoPE to balance efficiency and accuracy, while Apple's OpenELM family [127] employs layer-wise scaling and grouped query attention to optimize performance on low-powered devices. Other notable compact LLMs, such as Pythia [14], OPT [220], Qwen [12, 207], and MobileLLM [117] also benefit from similar optimizations, thereby reinforcing the growing potential of small-scale models for edge deployment.

While these methods significantly reduce parameter counts and resource demands, their development and optimization are predominantly executed by large organizations with substantial computational resources and domain expertise [127]. However, individual researchers can still benefit by using pre-trained compact LLMs from Hugging Face.¹ These models, often with optimized weights and configurations, can be deployed on edge devices for efficient inference, lowering the entry barrier and enabling broader access to advanced AI capabilities in resource-limited settings [14, 176].

3.6 Comparative Analysis and Selection

The efficacy of offline pre-deployment techniques for LLMs depends on the model architecture, deployment constraints, and performance requirements. The subsequent comparison provides an analysis to guide practitioners in selecting the most appropriate technique for their use case.

The selection of appropriate pre-deployment techniques should be informed by a comprehensive analysis of the deployment environment, hardware constraints, performance requirements, and LLM characteristics. For example, scenarios with strict memory constraints but moderate computational resources may benefit from weight-only quantization (e.g., LLM.int8() [32]) and structured pruning (e.g., CoFi [197]). Conversely, applications requiring maximum compression with some tolerance for accuracy loss might employ weight-activation co-quantization (e.g., SmoothQuant [199]) and aggressive unstructured pruning (e.g., Movement Pruning [150]). Knowledge distillation techniques such as MiniLM [189] or Lion [82] are suitable for transferring knowledge from larger to smaller models, with the choice between white-box or black-box approaches depending on teacher model accessibility. For models with large, redundant matrices, low-rank approximation methods like ALBERT [99] or LoSparse [109] can effectively reduce model size while preserving performance. In practice, combining multiple techniques often yields better results. ZeroQuant [211] demonstrates the efficacy of integrating quantization with knowledge distillation. NVIDIA's Nemotron-4 4B [134] exhibits high performance for on-device inference, which was pruned and distilled from Nemotron-4 15B [140]. DeepSeek-R1 [58] distills knowledge from compact models like Llama 3 [38], forming an edge-optimized 1.5B model further refined via large-scale reinforcement learning for mathematics, code generation, and complex reasoning, significantly enhancing inference capabilities.

When feasible, developing inherently efficient architectures such as LLaMA [176] or Gemma [169] may provide optimal performance within strict resource constraints. These approaches, typically developed by major tech companies with access to high-quality data and ample computing resources, demonstrate that careful architecture design and training strategies can yield highly efficient models without extensive post-training compression.

¹Hugging Face: <https://huggingface.co/>

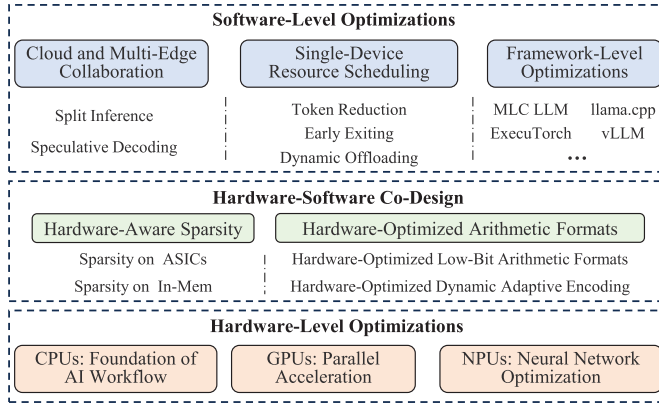


Fig. 8. A top-down view of runtime inference optimizations. (In-Mem: Computing/PIM).

4 Online Runtime Inference Optimizations

We previously discussed offline optimization techniques in the pre-deployment phase, including model compression techniques and pre-training compact models for edge devices. This section shifts to runtime optimizations for efficient inference of LLMs directly on edge devices. As shown in Figure 8, these optimizations are categorized into three areas: first, **Software-Level Optimizations**, which focuses on algorithmic strategies, resource scheduling, and framework optimizations independent of hardware; second, **Hardware-Software Co-Design**, which involves co-optimized solutions that leverage specific hardware features by adapting techniques such as sparsity and quantization to be hardware-friendly; and third, **Hardware-Level Optimizations**, which highlights innovations in hardware designed to improve the performance of LLMs.

4.1 Software-Level Optimizations

Software-level strategies can be categorized into Cloud and Multi-Edge Collaboration, Single-Device Resource Scheduling, and Framework-Level Optimizations, all of which focus on optimizing software algorithms, systems, frameworks, or engines without modifications tailored to specific hardware.

4.1.1 Cloud and Multi-Edge Collaboration. Collaborative computing across cloud and edge devices plays a critical role in enhancing LLM performance by distributing computational workloads efficiently [37, 187]. Although it has seen extensive use in tasks like video analytics [7, 75, 80, 120, 122, 137, 225], multi-device and cloud-edge collaboration for LLM computation remains in its infancy and generally follows one of two approaches: split inference or speculative decoding, as illustrated in Figure 9.

Split inference optimizes resource utilization and accelerates inference by partitioning computation across cloud and edge devices. PETALS [17] aggregates idle computational resources from diverse sources to ensure robust performance under dynamic network conditions. Voltage [70] improves throughput by distributing transformer layers across edge devices, achieving linear speed-ups through parallel computation. EASTER [60] addresses reliability challenges with adaptive partition strategies that maintain performance despite device failures. LinguaLinked [227] enhances split inference for mobile scenarios by aligning model segments with trusted device capabilities, enabling efficient data exchange. Approaches such as rewardless guidance [63] and Hepti [102] further optimize offloading decisions and dynamically adapt workloads to network and resource

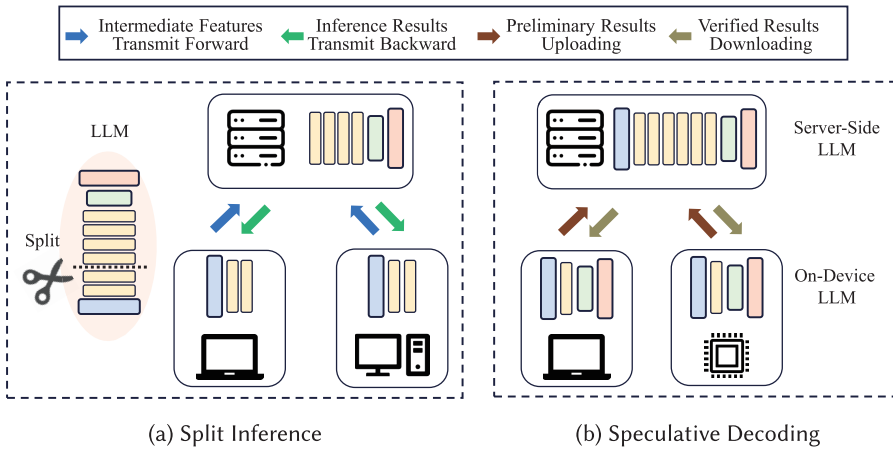


Fig. 9. Illustrations of cloud and multi-edge collaboration for on-device LLMs.

conditions. Building on these advancements, Zhang et al. [223] propose a tree-search framework to efficiently manage request batching and resource allocation, demonstrating the potential for adaptive and efficient cloud-edge collaboration.

Speculative decoding [103] addresses a key limitation of split inference, where the transmission of intermediate features often incurs significant communication overhead. In contrast, speculative decoding uploads preliminary results from edge devices and downloads only the verified results from a cloud-side LLM. This streamlined communication mechanism offers a practical alternative for scenarios where bandwidth or latency constraints are critical. For example, SpecTr [161] uses lightweight models on edge devices to propose token drafts, enabling parallel validation by larger cloud models. Tabi [191] uses calibrated confidence scores to decide whether to upload tokens to the cloud for verification. Built atop speculative decoding, EdgeLLM [201] features a branch navigation and self-adaptive fallback strategy, enabling fast and accurate token generation.

To provide a concise overview of the advancements in cloud and multi-edge collaboration, Table 1 summarizes the primary literature. The table highlights the key methodologies, objectives, and application scenarios of each work, offering a comparative perspective on the progress in split inference and speculative decoding. By integrating these approaches, future research can address challenges such as latency, reliability, and resource constraints more effectively.

4.1.2 Single-Device Resource Scheduling. Efficient scheduling for single-device inference builds upon a layered approach, where different techniques tackle optimization challenges from complementary perspectives. As illustrated in Figure 10, recent advancements can be categorized into token reduction, early exiting, and dynamic offloading.

Token reduction focuses on reducing the computational load by selectively eliminating unnecessary tokens or inputs. For example, PoWER-BERT [51] eliminates non-critical word vectors. Length-Adaptive Transformer [88] adjusts input sequence lengths dynamically using a dropout variant called LengthDrop. LTP [91] prunes less significant tokens based on attention scores. Additionally, LLMingua [79] compresses prompts iteratively at the token level, and AutoCompressors [29] reduce long context windows into more compact summary vectors. These techniques collectively aim to minimize the input size, ensuring that only the most important data is processed, thereby optimizing memory usage and reducing the computational burden on edge devices.

Early exiting terminates inference once a predefined confidence threshold is met, thereby reducing unnecessary computations during the model's forward pass. This technique can further

Table 1. Summary of the Primary Literature in Cloud and Multi-edge Collaboration

Techniques	Works	Features
Split Inference	PETALS [16]	Utilizes distributed idle computational resources to ensure robust performance under dynamic network conditions.
	Voltage [70]	Achieves linear speed-ups by distributing transformer layers across edge devices, leveraging parallel computation.
	EASTER [60]	Implements adaptive partition strategies to sustain performance despite device failures.
	LinguaLinked [227]	Optimizes split inference by aligning model segments with trusted mobile device capabilities for efficient data exchange.
	Hepti [102]	Dynamically adapts workload distribution and optimizes offloading decisions based on network and resource conditions.
Speculative Decoding	SpecTr [161]	Employs lightweight models on edge devices to generate token drafts, enabling parallel validation by larger cloud models.
	Tabi [191]	Utilizes calibrated confidence scores to decide token uploads for the cloud verification, balancing efficiency and accuracy.
	EdgeLLM [201]	Features a branch navigation and self-adaptive fallback strategy, enabling fast and accurate token generation.

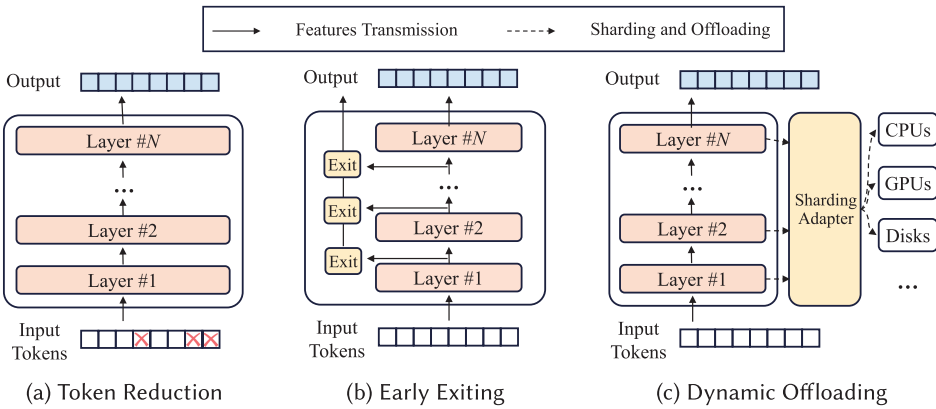


Fig. 10. Illustrations of single-device resource scheduling.

benefit from the reduced input size. For example, PABEE [231] enhances the efficiency and robustness of pre-trained language models by integrating internal-classifiers at each layer. MPEE [93] combines horizontal and vertical early exit strategies for adaptive inference. FREE [11] proposes a shallow-deep module to synchronize the decoding process of the current token with previously stacked early-exit tokens. ConsistentEE [216] integrates reinforcement learning to determine the optimal early exit points, balancing inference speed with result accuracy. Techniques like LeeBERT [233] and FastBERT [116] enable adaptive computation via early exiting with layer-wise confidence evaluation, ensuring efficient resource utilization while maintaining output quality.

Dynamic offloading further optimizes resource utilization by distributing tasks across different processing units. Techniques like STI [59] introduce elastic pipelining, where model components are dynamically sharded and assigned to available resources. FlexGen [154] further refines

Table 2. Summary of the Primary Literature in Single-device Resource Scheduling

Techniques	Works	Features
Token Reduction	PoWER-BERT [51]	Eliminates non-critical word vectors to reduce computation while preserving accuracy.
	Length-Adaptive Transformer [88]	Dynamically adjusts input sequence lengths using a dropout-based technique called LengthDrop [88].
	LTP [91]	Prunes less significant tokens by leveraging attention scores to optimize inference efficiency.
	LLMLingua [79]	Compresses prompts iteratively at the token level to streamline input processing.
	AutoCompressors [29]	Reduces long context windows into more compact summary vectors for efficient processing.
Early Exiting	PABEE [231]	Improves the efficiency and robustness of pre-trained language models by integrating internal classifiers at each layer.
	MPEE [93]	Combines horizontal and vertical early exit strategies, enabling a more adaptive and efficient inference process.
	ConsistentEE [216]	Utilizes reinforcement learning to identify optimal early exit points, balancing inference speed with output accuracy.
	FREE [11]	Introduces a shallow-deep module to sync current token decoding with earlier early-exit tokens.
Dynamic Offloading	STI [59]	Implements elastic pipelining, dynamically sharding model components and assigning them to available resources.
	FlexGen [154]	Leverages a hybrid memory architecture, combining GPU, CPU, and disk memory with intelligent I/O scheduling to handle computationally intensive tasks.
	LLM in a Flash [4]	Stores model parameters in flash memory and dynamically loads them into DRAM for efficient inference.

this concept by leveraging a hybrid memory model, utilizing GPU, CPU, and disk memory with intelligent I/O scheduling for offloading computationally intensive tasks. LLM in a Flash [4] stores the model parameters in flash memory and dynamically loads them into DRAM for inference. They ensure balanced workloads, ensuring that edge devices can efficiently handle large-scale LLM inference with minimal bottlenecks.

Recent studies, such as MELting Point [100], offer insights beyond traditional benchmarking by evaluating performance, memory, and energy demands across various model sizes and devices. These analyses reveal critical bottlenecks in computational efficiency, quality of experience, and accuracy, providing a foundation for advancements in algorithms and hardware. As summarized in Table 2, they enhance understanding of single-device resource scheduling optimization, addressing challenges in computational efficiency and resource allocation for edge inference tasks.

4.1.3 Framework-Level Optimizations. To meet the performance and portability demands of edge computing, framework-level optimizations focus on lightweight frameworks, libraries, and engines specifically designed. For instance, PyTorch, widely adopted for its flexibility and ecosystem support, has also extended its capabilities to edge computing with ExecuTorch [142], enabling efficient LLM inference through optimized execution plans for low-latency scenarios.

Building on this, DNNFusion [131] optimizes mobile execution with advanced operator fusion, combining graph rewriting and fusion plan generation. SmartMem [132] reduces memory over-

Table 3. Comparison of Popular Edge LLM Frameworks, Including Examples of Related Works That Have Optimized or Experimentally Deployed These Frameworks

Name	Description	Mobile Support	Desktop Support	Related Works
MLC LLM [171]	Comprehensive LLM deployment framework leveraging machine learning compilation techniques.	✓	✓	OmniQuant [151], MobileLLM [117], MELTing Point [100].
llama.cpp [49]	Efficient C/C++ implementation for LLM inference.	✓	✓	EdgeLLM [201], MELTing Point [100].
ONNX Runtime [128]	A cross-platform machine-learning model accelerator, with a flexible interface to integrate hardware-specific libraries.	✓	✓	EASTER [60].
ExecuTorch [142]	On-device LLM across mobile and edge for PyTorch.	✓	✓	Llama 3.2 [6].
vLLM [95]	Utilize key-value cache stored in non-contiguous paged memory to efficiently inference.	✗	✓	AWQ [112], GPTQ [45], QLLM [114].
Neural Compressor [73]	State-of-the-art low-bit quantization and sparsity for runtime LLMs.	✗	✓	SmoothQuant [199], AWQ [112], GPTQ [45], SparseGPT [44].
TensorRT-LLM [133]	Advanced library for optimizing LLM inference powered by NVIDIA.	✗	✓	SmoothQuant [199], AWQ [112].
MLX [62]	Apple-developed array framework tailored for machine learning research on Apple silicon.	✗	✓	OpenELM [127].

head by eliminating redundant layout transformations and selecting optimal memory layouts. PowerInfer [157] enhances runtime efficiency by preloading frequently activated neurons onto the GPU and processing less active neurons on the CPU to minimize memory and data transfer overhead. Furthermore, vLLM [95] introduces PagedAttention, inspired by virtual memory management, to segment attention key-value caches into blocks, enabling efficient memory sharing across sequences and requests. This approach improves memory efficiency while supporting quantization methods and optimized GPU kernels, making vLLM versatile for edge use.

The framework-level optimization ecosystem, as summarized in Table 3, includes tools tailored for LLM deployment on edge devices. The table categorizes frameworks by their descriptions, platform compatibility, and references to related works, helping readers identify suitable options and learn from previous implementations.

4.2 Hardware-Software Co-Design

Hardware-software co-design is a cross-disciplinary approach that integrates hardware and software optimization to enhance the performance, energy efficiency, and scalability of machine learning models. Unlike isolated optimization strategies, co-design aligns algorithmic innovations with hardware-specific capabilities, addressing the unique constraints of edge devices, such as limited power, memory, and computational resources. This synergy is essential for achieving high-efficiency inference in LLMs on resource-constrained platforms.

To provide a comprehensive overview of co-design optimizations, Table 4 compares software features and hardware platforms, highlighting the inference speedup and energy efficiency achieved by various approaches. Techniques are categorized into two primary areas: hardware-aware sparsity and hardware-optimized arithmetic formats. These classifications reflect different optimization focuses, enabling a clearer understanding of their respective contributions.

Table 4. Comparison of Edge LLM-based Co-design Optimization Techniques

Type	Name	SW Features	HW Chips	Inference Speedup* or Energy Efficiency [†]
Hardware-Aware Sparsity	SpAtten[186]	Token-based sparsity and multi-head quantization.	ASIC 40 nm simulator	1095 $\times^*$, 406 $\times^{\dagger}$ Jetson Nano GPU
	Sanger[118]	Threshold-based pruning prediction.	ASIC 55 nm simulator	1.47 $\times^*$ SpAtten
	EdgeBERT[163]	Early exit prediction with pruning-aware optimizations	ASIC 12 nm simulator	53 $\times^{\dagger}$ Jetson Tegra X2 GPU
	TaskFusion [42]	Hardware-aware sub-task pruning with combined sparsity	ASIC 22 nm simulator	8.21 $\times^*$, 19.83 $\times^{\dagger}$ Jetson Nano GPU
	AccelTran[182]	Activation sparsity via dynamic cycle simulation.	ASIC 14 nm simulator	280 $\times^*$, 236 $\times^{\dagger}$ Apple M1 GPU
	4.60mm ² STP[165]	Entropy-based early exit and mixed-precision sparsity.	ASIC 12 nm tapeout	18.1 TFLOPS/W (FP4) [†]
	C-Transformer[90]	Hybrid pruning for spiking/non-spiking Transformers.	ASIC 28 nm tapeout	47.8 TOPS/W (INT8) [†]
	TransPIM[229]	Token-based dataflow optimization	In-Mem 65 nm simulator	114.9 $\times^*$, 666.6 $\times^{\dagger}$ RTX 2080 Ti GPU
	X-Former[158]	Attention pruning with projection-based sparsity	In-Mem 32 nm simulator	19.6 $\times^*$, 13 $\times^{\dagger}$ GTX 1060 GPU
	TranCIM[180]	Dynamic reconfiguration of streaming networks.	In-Mem 28 nm tapeout	20.5 TOPS/W (INT8) [†]
Hardware-Optimized Arithmetic Formats	MulTCIM[181]	Token pruning with reshaped attention matrices.	In-Mem 28 nm tapeout	101.1 TOPS/W (INT8) [†]
	GOBO[214]	Map weights to a dictionary of 3-/4-bit indexed representative floats.	ASIC 65 nm simulator	7 $\times^*$, 3 $\times^{\dagger}$ Tensor Cores-based accelerator
	Mokey[215]	Quantizing values to 4-bit index counting instead of floating-point multiplications and accumulations.	ASIC 65 nm simulator	9 $\times^{\dagger}$ GOBO
	OliVe[56]	Abfloat Format: An outlier-victim pair quantization strategy designed for sparse outlier distributions.	ASIC 22 nm simulator	4.5 $\times^*$, 4 $\times^{\dagger}$ GOBO
	AdaptivFloat[164]	HFINT Format: Dynamically adjusts exponent bias across network layers to optimize precision.	ASIC 16 nm simulator	1.09 $\times^{\dagger}$ INT8-based accelerator
	ANT[57]	Flint Format: Adaptive bit-width selection between float and int.	ASIC 28 nm simulator	4 $\times^*$, 3.33 $\times^{\dagger}$ AdaptivFloat

(In-Mem: Computing/PIM. SW: Software. HW: Hardware).

4.2.1 Hardware-Aware Sparsity. Hardware-aware sparsity focuses on integrating hardware-specific considerations into model design. This strategy can be further categorized into two primary approaches: sparsity on ASICs and sparsity on in-memory accelerators.

Sparsity on ASICs. ASIC-based accelerators achieve fine-grained control over model sparsity through custom circuit designs. Simulator-based methods explore early-stage designs: SpAtten [186] employs token pruning and a top-k ranking engine to prioritize token and head importance. Sanger [118] synergistically co-designs software that prunes attention matrices into dynamic structured patterns and hardware with a reconfigurable architecture. EdgeBERT [163] dynamically scales voltage and frequency, prunes networks, and quantizes floating points. TaskFusion [42] combines weight and activation sparsity with hardware-aware sub-task inference algorithms. AccelTran [182] enhances activation sparsity dynamically using a cycle-accurate simulator. Real-world tapeouts validate these designs: STP [165] optimizes latency and energy through mixed-precision computation and fine-grained power management. Similarly, C-Transformer [90] integrates spiking and non-spiking Transformers, achieving high sparsity and hardware utilization, illustrating the potential of hybrid architectures.

Sparsity on In-Memory Accelerators. In-memory accelerators address data movement challenges by embedding computation within memory arrays. **Processing-in-Memory (PIM)** augments conventional memory hierarchies with compute units near memory, while **Computing-in-Memory (CIM)** embeds compute capabilities directly into memory cells for fine-grained operations. PIM-based sparsity designs like TransPIM [229] use token-based dataflows and high-bandwidth memory to minimize communication overhead, and X-Former [158] combines a software-level attention engine with nonvolatile memory and CMOS tiles. CIM-based sparsity designs, such as TranCIM [180], dynamically reconfigure streaming networks and bitline-transpose structures to reduce complexity. MultCIM [181] addresses hybrid sparsity with runtime token pruning and attention matrix reshaping, improving scalability and efficiency.

Comparative Insights. ASIC accelerators excel in throughput optimization through fine-grained sparsity, whereas in-memory accelerators focus on reducing data movement with token-based designs. Together, these approaches demonstrate the transformative potential of hardware-aware sparsity in edge scenarios.

4.2.2 Hardware-Optimized Arithmetic Formats. Hardware-optimized arithmetic formats optimize inference performance by balancing computational accuracy with hardware efficiency. These formats employ two synergistic strategies: low-bit arithmetic and dynamic adaptive encoding.

Hardware-Optimized Low-Bit Arithmetic Formats. Low-bit arithmetic formats reduce the numerical precision of model parameters to fixed low-bit representations, tailored for specialized hardware accelerators. For example, GOBO [214] reduces 32-bit floating-point parameters to just 3 bits, significantly lowering power consumption. Mokey [215] quantizes parameters to 4-bit representations, enabling area- and energy-efficient hardware acceleration. OliVe [56] introduces an outlier-victim pair quantization approach, which sacrifices normal values to accommodate outliers, enabling more efficient memory alignment and boosting performance.

Hardware-Optimized Dynamic Adaptive Encoding. Dynamic adaptive encoding adjusts numerical precision based on runtime requirements, providing greater flexibility compared to fixed low-bit formats. AdaptivFloat [164] dynamically adjusts tensor-wide exponent biases to maintain accuracy at low bit widths. ANT [57] introduces a flexible adaptive data format, Flint, which combines floating-point and integer precision for low-bit quantization with minimal hardware overhead.

Comparative Insights. Low-bit formats offer energy efficiency and predictable performance but may sacrifice accuracy, whereas adaptive encoding provides precision flexibility at the cost of added complexity. Combining both strategies can yield robust and adaptable hardware-software designs for on-device LLMs.

In conclusion, hardware-software co-design represents a critical avenue for optimizing LLMs on edge devices. However, much of the research remains in the simulation phase, with limited real-world deployment on general-purpose heterogeneous platforms. By summarizing these approaches, we aim to highlight how tailored software optimizations can exploit hardware-specific features to outperform similar algorithms on existing hardware platforms. This discussion serves as a guide for future developments in deploying efficient and scalable LLMs in resource-constrained environments.

4.3 Hardware-Level Optimizations

The development of hardware architectures is essential for deploying LLMs on devices, enhancing inference speed and energy efficiency. Table 5 outlines prominent commercial hardware chips, emphasizing AI performance and related studies. The AI performance column highlights key metrics of **System-on-Chips (SoCs)**, CPUs, and GPUs, while the Related Works column helps readers

Table 5. Comparison of Popular Commercial Hardware Devices for On-device LLMs

Hardware	Chip	Type	AI Performance	Related Works
SoCs (CPU + iGPU + NPU)	Samsung Exynos	Mobile	Exynos 2400 equips with NPU and DSP AI Engine [41]	Llama 3.2 [6]
	Google Tensor	Mobile	Tensor G4 can run Gemini Nano [168] with multi-modality [50]	MELting Point [100], Llama 3.2 [6], Gemini Nano [168], MobileLLM [117]
	Apple A	Mobile	A16 Bionic features 16-core Neural Engine with 17 TOPS [8]	MobileLLM [117], Llama 3.2 [6], MELting Point [100], SpQR [33]
	Apple M	Laptop	M2 Ultra features a 32-core Neural Engine with 31.6 TOPS [9]	MELting Point [100], LLM in a Flash [4]
	Qualcomm Snapdragon	Mobile	Snapdragon 8 Gen 3 achieves up to 20 tokens/sec for LLM inference [172]	EdgeLLM [201], MobileLLM [117], MobileBERT [162], Llama 3.2 [6], MELting Point [100], Agile-Quant [153]
CPUs	Intel Core	Desktop	Intel i9-13900k combines Performance and Efficient cores with Deep Learning Boost [74]	SmoothQuant [199], EASTER [60], AQLM [40], Neural Compressor [73], FREE [11]
	ARM Cortex-A72 (on RPi 4/4B)	IoT	RPi 4B handles lightweight LLM inference efficiently with quantization and token pruning [153]	MELting Point [100], Hepti [102], Agile-Quant [153]
GPUs	NVIDIA GeForce Laptop GPU	Laptop	RTX 4070 Laptop GPU achieves 321 TOPS with 8 GB memory [46]	AWQ [112]
	NVIDIA GeForce Desktop GPU	Desktop	RTX 4090 offers 1321 TOPS with 24 GB memory [47]	AWQ [112], SpQR [33], OWQ [101], AQLM [40], TensorRT-LLM [133], QuIP# [179], LLM in a Flash [4], Drone [25]
	NVIDIA Jetson Modules GPU	IoT	Jetson AGX Orin delivers 275 TOPS with 64 GB memory [34]	EdgeLLM [201], MELting Point [100], STI [59], Beyond the Cloud [223], TensorRT-LLM [133]

(RPi: Raspberry Pi).

identify suitable hardware and explore relevant optimization strategies from previous implementations.

4.3.1 CPUs: Foundation of AI Workflow. CPUs remain fundamental in AI workflow management, providing essential flexibility and system-level integration. Recent advancements have expanded their LLM potential for LLM inference. For example, Arm Cortex A72 on Raspberry Pi 4B (8 GB RAM) achieves low latency inference of LLaMA-7B [176] when paired with Agile-Quant [153]. Intel i9-13900k [74] enhanced by AQLM [40], provides efficient inference for the LLaMA-2 model [177].

However, CPUs' architecture, optimized for sequential tasks, struggle with the parallel computations needed for LLM inference, especially in edge scenarios requiring real-time performance and energy efficiency. To address this, modern CPUs are often combined with specialized accelerators (e.g., GPUs and NPUs) in SoCs like Apple's M-series or A-series SoCs [8, 9], Google's Tensor G4 [50], and Qualcomm Snapdragon chips [172]. These heterogeneous architectures aim to balance flexibility and performance by integrating general-purpose and parallel processing units. However, managing data flow and power consumption across these components remains a challenge.

4.3.2 GPUs: Parallel Acceleration. To address the inefficiency of CPUs, GPUs have been developed as highly parallel processors to accelerate computationally intensive tasks in edge computing. Equipped with hundreds or thousands of smaller cores, GPUs enable massive parallel computation for LLM inference. Modern edge GPUs, such as the NVIDIA Jetson series [34], feature specialized hardware such as Tensor Cores, optimized for matrix operations prevalent in LLM inference tasks.

For example, Yuan et al. [213] demonstrate the feasibility of LLM inference on NVIDIA Jetson ORIN NX [34].

However, GPUs face significant power consumption challenges in edge scenarios, often necessitating hybrid models that distribute intensive computations between cloud and local resources. This strategy balances high-performance inference with the energy constraints of mobile and edge devices. Additionally, GPUs are frequently integrated with CPUs and NPUs in heterogeneous systems, complicating integration and resource management. Effective scheduling and task offloading among these processors are critical challenges in edge deployment, requiring advanced software frameworks to ensure efficient collaboration.

4.3.3 NPUs: Neural Network Optimization. NPUs are specialized accelerators designed to optimize neural network computations, offering substantial improvements in performance and energy efficiency for edge LLM inference. By employing low-precision arithmetic (e.g., INT8) and highly parallelized architectures, NPUs enable real-time inference with minimal power consumption. Notable examples include Apple's Neural Engine in the M-series or A-series chips [8, 9] and Qualcomm's AI engines in Snapdragon processors [172], which boost on-device LLM capabilities and reduce cloud dependency.

However, NPUs face key limitations that restrict their broader applicability. They are optimized for a limited set of neural network operators, making them incompatible with many modern LLM architectures. This often forces fallback strategies like CPU-NPU co-processing, which can offset performance gains. Additionally, the rapid evolution of neural architectures further complicates matters, as NPUs struggle to keep up with model diversity and complexity, requiring extensive adaptations or failing to execute some models entirely [213]. In heterogeneous hardware environments, effective integration of NPUs with CPUs and GPUs is essential, as NPUs' specialization may necessitate offloading tasks to general-purpose processors, complicating resource management.

In conclusion, online runtime inference optimizations enhance LLM performance on edge devices through software-level optimizations, hardware-software co-design, and hardware-level enhancements. These methods complement offline pre-deployment techniques, forming a holistic approach to on-device LLM optimization. Offline techniques reduce computational complexity and memory footprint via pre-training and fine-tuning, while runtime strategies focus on efficient resource utilization, dynamic adaptation, and scalability. Integrating both phases is crucial for high-performance LLMs on resource-constrained platforms.

5 On-Device LLM-Based Applications

Leveraging compact models and runtime optimizations, on-device LLMs enable efficient, low-latency, and privacy-preserving AI in edge environments. As shown in Figure 11, on-device LLM-based application systems span personal, enterprise, and industrial domains.

5.1 Personal-Use Applications

On-device LLMs enable ubiquitous AI experiences in edge environments, driving advancements in personal assistants, healthcare assistants, and companion robots.

5.1.1 Personal Agents. On-device LLMs are transforming personal digital assistants by providing low-latency, privacy-preserving, and always available support tailored to individual needs. AutoDroid [194] automates daily tasks such as food ordering and health tracking, leveraging multi-granularity token pruning techniques to operate efficiently within the limited computational budget of mobile devices. Similarly, LlamaTouch [218] redefines mobile UI task automation evaluation, while CoCo-Agent [125] enhances GUI automation for complex interactions with real-world en-

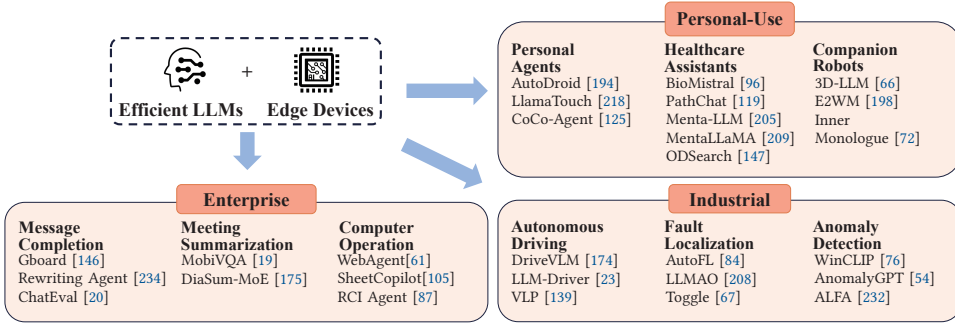


Fig. 11. Illustrations of on-device applications for LLMs.

vironments. These advancements streamline personal productivity by automating routine tasks, including e-mail management, photo editing, and scheduling [194].

5.1.2 Healthcare Assistants. In healthcare, on-device LLMs demonstrate versatility by providing domain-specific support in areas like clinical decision-making and mental health analysis. BioMistral [96], a 7B biomedical LLM, exemplifies the synergy between domain-specific fine-tuning and quantization to achieve low-latency inference. PathChat [119] integrates visual and natural language inputs through a multimodal projector module connecting a vision encoder to the LLaMA-2 model [177], transforming pathology education and research. Mental-LLM [205] and MentaLLaMA [209] focus on interpretable mental health analysis on social media platforms. On-device LLMs also enable wearable applications such as ODSearch [147], which provides a natural language interface for fitness tracker data, delivering near real-time search capabilities.

5.1.3 Companion Robots. On-device LLMs are transforming companion robots by enabling rapid, context-aware interactions and excelling in both verbal and non-verbal tasks. These robots can build connections through dialogue, negotiate with users, generate contextually appropriate responses, and execute actions requiring physical interaction. 3D-LLM [66] integrates 3D environment reasoning and planning, enabling robots to analyze spatial contexts and make informed decisions. E2WM [198] augments multiple compact LLMs (e.g., LLaMA [176] and OPT [220]) with embodied knowledge and skills, improving adaptability in dynamic environments. Inner Monologue [72] enhances these capabilities by allowing 1.3B InstructGPT [136] to reason over natural language feedback for better planning and execution in embodied tasks.

5.2 Enterprise Applications

On-device LLMs boost enterprise privacy and cost-efficiency, improving tasks like message completion, meeting summaries, and computer operations.

5.2.1 Message Completion. On-device LLMs streamline enterprise communication by generating context-aware responses, improving productivity through automated replies. For instance, Google's Gboard [146] integrates Gemini Nano [168] for real-time, context-sensitive suggestions, utilizing a cloud-edge framework where the cloud handles complex tasks and the edge manages lightweight interactions. Zhu et al. [234] propose a mobile-centric text rewriting LLM with efficient dataset generation and cascading mechanisms, while ChatEval [20] autonomously evaluates text quality, aligning closely with human judgment.

5.2.2 Meeting Summarization. In meetings, on-device LLMs are enabling automated summarization of key discussions and decisions, offering a privacy-preserving alternative to cloud-based

solutions while maintaining high efficiency. For example, MobiVQA [19] delivers an efficient on-device visual question-answering system with attention-based early exit and question-aware pruning techniques. Tian et al. [175] propose a MoE framework for role-oriented dialogue summarization, enhancing contextual nuance in business settings.

5.2.3 Computer Operation. On-device LLMs facilitate natural language interfaces for computer operations, automating tasks and reducing user cognitive load. WebAgent [61] enables task execution on websites through real-time programming. SheetCopilot [105] simplifies spreadsheet interactions by translating natural language commands into actionable tasks. The RCI agent [87] uses LLMs to autonomously complete tasks on a computer using a keyboard and mouse, iteratively refining its output to improve performance. These systems leverage cloud-edge collaboration, with the cloud handling intensive planning and the edge executing lightweight, privacy-preserving operations.

5.3 Industrial Applications

On-device LLMs enhance industrial systems by enabling real-time analysis and decision-making in applications like autonomous driving, fault localization, and anomaly detection while reducing network overhead and improving response times.

5.3.1 Autonomous Driving. On-device LLMs enhance autonomous driving by integrating linguistic understanding with navigation and decision-making. DriveVLM-Dual [174] uses a visual LLM to interpret urban environments and plan routes via natural language, efficiently deployed on NVIDIA Orin platforms [34]. LLM-Driver [23] leverages object-level vector inputs for explainable driving action prediction. Similarly, VLP [139] strengthens both contextual understanding and memory foundation in self-driving systems. These advancements demonstrate the transformative potential of LLMs in autonomous driving.

5.3.2 Fault Localization. In software fault localization, LLM-based techniques offer significant improvements over traditional machine learning methods. AutoFL [84] generates bug explanations and identifies fault locations by navigating software repositories and mitigating LLM context length constraints. LLMAO [208] detects logic and security vulnerabilities without relying on extensive program analysis or test cases, outperforming deep learning-based methods. Toggle [67] employs tailored prompts and an adjustment module to localize and fix bugs at the token level, enabling fine-grained debugging.

5.3.3 Anomaly Detection. On-device LLMs also advance industrial anomaly detection by leveraging multi-modal capabilities and efficient feature aggregation [230]. WinCLIP [76] supports zero-shot and few-shot anomaly classification and segmentation using state word ensembles and prompt templates. AnomalyGPT [54] employs large vision-language models to identify anomalies directly, removing the need for manual threshold adjustments while showcasing strong few-shot learning performance. ALFA [232] addresses zero-shot visual anomaly detection by generating adaptive prompts, reducing semantic ambiguities, and fusing local pixel-level information for precise localization.

In summary, on-device LLMs use both offline and online optimization techniques to enable efficient, private, and responsive edge AI applications. Offline methods like quantization, pruning, and compact model design (e.g., Gemini Nano [168], BioMistral [96]) reduce model size and computational demands, making them suitable for resource-constrained devices. These are supported by runtime optimizations such as cloud-edge collaboration (e.g., WebAgent [61]), early exiting (e.g., MobiVQA [19]), and hardware acceleration (e.g., DriveVLM-Dual [174] on NVIDIA Orin [34]). To-

gether, these strategies highlight the synergy between offline compression and runtime efficiency, driving innovation across personal, enterprise, and industrial applications.

6 Future Directions and Open Challenges

The rapid deployment of LLMs on mobile edge devices offers opportunities but faces key limitations. First, transitioning from centralized servers to decentralized edge nodes brings challenges due to device heterogeneity, including variations in computational power, communication latency, and fault tolerance [81, 120]. Second, many hardware-software co-design techniques show promise in simulations but often fail in real-world heterogeneous deployments [193]. Third, on-device LLMs struggle with complex scenarios requiring multi-hop reasoning, such as dynamic multi-agent interactions or real-time adaptation in personalized applications [26, 202, 203, 210]. These limitations hinder efficient inference and complicate the design of robust edge LLM-based systems. In the following, we explore key research directions and open challenges to address these limitations:

Compact LLM Architecture Development. Traditional Transformer-based models [183] are computationally intensive and impractical for edge devices due to their high memory and processing demands. To address edge device heterogeneity and reduce resource consumption, Mamba [52] introduces linear computational scaling through selective mechanisms, while Jamba [111] combines Transformer and Mamba layers for better adaptability. However, deploying these models on edge devices faces challenges [52, 111]: (1) their selective state space architecture demands substantial computational resources and memory bandwidth; (2) their recurrent nature inherently limits parallelization and hardware acceleration compared to transformers; and (3) optimizing them through quantization and pruning is complex due to interactions between selective state spaces and nonlinear components.

Innovative Edge-Cloud Collaboration. Cloud-edge collaboration can mitigate the limitation of insufficient memory and computing power of individual edge devices, which has seen progress with techniques like split inference [17] and speculative decoding [103]. However, distributed decoding strategies for edge deployment remain underexplored. Techniques such as disaggregated prefill and decoding [228], which separate tasks to optimize resource allocation, show potential for reducing inter-phase interference. However, to achieve efficient edge-cloud collaboration, it is crucial to address the challenges posed by fault tolerance and communication costs.

Heterogeneous Deployment with Simulation Techniques. Many hardware-software co-design approaches [56, 163, 186] for edge LLM inference have been evaluated in simulation or on specific ASIC prototypes, but the limitations are Cannot be used effectively on general-purpose hardware platforms. Future research should focus on improving scalability with heterogeneous hardware platforms, including CPUs, GPUs, and NPUs [213]. By optimizing for real-world scenarios, these methods can achieve faster, more energy-efficient LLM inference while addressing the challenges of dynamic workloads and diverse hardware configurations.

Graph-based LLM Development. On-device LLMs face significant limitations in handling graph-related inference tasks, which are critical for complex relationships and multi-hop reasoning in domains such as social networks, biology, and transportation. Recent approaches, including GraphRAG [39] for structured data extraction, GraphGPT [167] for graph interpretation through structural knowledge alignment, and GraphWiz [24] can address the multi-hop reasoning and complex scenarios limitation by extracting and interpreting graph-based data. However, the resource constraints of edge devices limit the scalability of these methods.

Multi-Agent Collaboration. While many on-device LLM applications, such as personal assistants [194, 218] and companion robots [66, 72], excel in single-agent tasks, they often struggle to operate effectively in dynamic and complex environments. To address this limitation, collabora-

tive intelligence among heterogeneous agents leverages dynamic orchestration frameworks to enable multi-agent coordination, enhancing task performance beyond single-model systems [75, 123]. However, deploying such frameworks on edge devices remains challenging due to constraints in communication bandwidth, latency, and energy efficiency [122, 173].

Continual Learning and Personalization. On-device LLMs encounter significant challenges in real-time adaptation and personalization, especially under resource constraints. Continual learning provides a promising approach by enabling models to dynamically adapt while addressing issues such as catastrophic forgetting [13, 121, 225]. For example, Interactive Continual Learning [143] utilizes ongoing interactions to enhance model performance in complex scenarios. However, the limited parameter capacity of edge models restricts their ability to acquire new knowledge, and fine-tuning on-device demands substantial computational resources, complicating efficient adaptation [86, 137].

7 Conclusion

This survey provides a comprehensive review of recent advancements in enabling on-device LLMs, systematically exploring Offline Pre-Deployment Model Design Techniques, Online Runtime Inference Optimizations, and On-Device LLM-Based Applications. These components form a cohesive optimization pipeline: pre-deployment methods such as quantization and pruning create compact, efficient models; runtime techniques ensure adaptability and performance across heterogeneous environments; and diverse applications showcase the practical impact of edge LLMs. By addressing key challenges in efficiency and scalability, this survey offers valuable insights for researchers and practitioners, paving the way for accessible, sustainable AI solutions that unlock the full potential of LLMs.

References

- [1] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. Retrieved from <https://arxiv.org/abs/2404.14219>
- [2] Samira Abnar and Willem Zuidema. 2020. Quantifying attention flow in transformers. In *Proceedings of the ACL*.
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the EMNLP*.
- [4] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. Llm in a flash: Efficient large language model inference with limited memory. In *Proceedings of the ACL*.
- [5] Yongqi An, Xu Zhao, Tao Yu, Ming Tang, and Jinqiao Wang. 2024. Fluctuation-based adaptive structured pruning for large language models. In *Proceedings of the AAAI*.
- [6] Yuvanesh Anand, Zach Nussbaum, Brandon Duderstadt, Benjamin Schmidt, and Andriy Mulyar. 2024. Introducing Llama 3.2. Retrieved December 5, 2024 from https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_2/
- [7] Ganesh Ananthanarayanan, Victor Bahl, Landon Cox, Alex Crown, Shadi Noghahi, and Yuanhao Shu. 2019. Demo: Video analytics - killer app for edge computing. In *Proceedings of the ACM MobiSys*.
- [8] Apple. 2023. Apple Debuts iPhone 15 and iPhone 15 Plus. Retrieved December 3, 2024 from <https://www.apple.com/newsroom/2023/09/apple-debuts-iphone-15-and-iphone-15-plus/>
- [9] Apple. 2023. Apple Introduces M2 Ultra. Retrieved December 3, 2024 from <https://www.apple.com/newsroom/2023/06/apple-introduces-m2-ultra/>
- [10] Saleh Ashkboos, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. 2024. SliceGPT: Compress large language models by deleting rows and columns. In *Proceedings of the ICLR*.
- [11] Sangmin Bae, Jongwoo Ko, Hwanjun Song, and Se-Young Yun. 2023. Fast and robust early-exiting framework for autoregressive language models with synchronized parallel decoding. In *Proceedings of the EMNLP*.
- [12] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. Retrieved from <https://arxiv.org/abs/2309.16609>

- [13] Romil Bhardwaj, Zhengxu Xia, Ganesh Ananthanarayanan, Junchen Jiang, Yuanchao Shu, Nikolaos Karianakis, Kevin Hsieh, Paramvir Bahl, and Ion Stoica. 2022. Ekya: Continuous learning of video analytics models on edge compute servers. In *Proceedings of the USENIX NSDI*.
- [14] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. 2023. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the ICML*.
- [15] Yelysei Bondarenko, Markus Nagel, and Tijmen Blankevoort. 2021. Understanding and overcoming the challenges of efficient transformer quantization. In *Proceedings of the EMNLP*.
- [16] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Maksim Riabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. 2023. Petals: Collaborative inference and fine-tuning of large models. In *Proceedings of the ACL*.
- [17] Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin, and Colin Raffel. 2023. Distributed inference and fine-tuning of large language models over the internet. In *Proceedings of the NeurIPS*.
- [18] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Proceedings of the NeurIPS*.
- [19] Qingqing Cao, Prerna Khanna, Nicholas D. Lane, and Aruna Balasubramanian. 2022. MobiVQA: Efficient on-device visual question answering. In *Proceedings of the ACM UbiComp*.
- [20] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. 2024. ChatEval: Towards Better LLM-based evaluators through multi-agent debate. In *Proceedings of the ICLR*.
- [21] Arnav Chavan, Nahush Elele, and Deepak Gupta. 2024. Surgical feature-space decomposition of LLMs: Why, when and how? In *Proceedings of the ACL*.
- [22] Hongzhan Chen, Siyue Wu, Xiaojun Quan, Rui Wang, Ming Yan, and Ji Zhang. 2023. MCC-KD: Multi-CoT consistent knowledge distillation. In *Proceedings of the EMNLP*.
- [23] Long Chen, Oleg Sinavski, Jan Hünemann, Alice Karnsund, Andrew James Willmott, Danny Birch, Daniel Maund, and Jamie Shotton. 2024. Driving with llms: Fusing object-level vector modality for explainable autonomous driving. In *Proceedings of the IEEE ICRA*.
- [24] Nuo Chen, Yuhua Li, Jianheng Tang, and Jia Li. 2024. Graphwiz: An instruction-following language model for graph computational problems. In *Proceedings of the ACM SIGKDD*. 353–364.
- [25] Patrick Chen, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2021. DRONE: Data-aware low-rank compression for large NLP models. In *Proceedings of the NeurIPS*.
- [26] Tao Chen, Yongjie Yang, Xiaoran Fan, Xiuzhen Guo, Jie Xiong, and Longfei Shangguan. 2024. Exploring the feasibility of remote cardiac auscultation using earphones. In *Proceedings of the ACM MobiCom*.
- [27] Yuhao Chen, Yuxuan Yan, Qianqian Yang, Yuanchao Shu, Shibo He, and Jiming Chen. 2023. Confidant: Customizing transformer-based LLMs via collaborative edge training. Retrieved from <https://arxiv.org/abs/2311.13381>
- [28] Zeming Chen, Qiye Gao, Antoine Bosselut, Ashish Sabharwal, and Kyle Richardson. 2023. DISCO: Distilling counterfactuals with large language models. In *Proceedings of the ACL*.
- [29] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts. In *Proceedings of the EMNLP*.
- [30] CPU-Monkey. 2024. AI Performance (NPU) CPU Benchmark List. Retrieved July 18, 2024 from https://www.cpu-monkey.com/en/cpu_benchmark-ai_benchmark
- [31] Yubin Dai, Bin Qian, Yangkun Liu, Yuxuan Yan, and Yuanchao Shu. 2025. Eros: Real-time dense mapping made easy on mobile devices. In *Proceedings of the ACM HotMobile*.
- [32] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8() 8-bit matrix multiplication for transformers at scale. In *Proceedings of the NeurIPS*.
- [33] Tim Dettmers, Ruslan A. Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. 2023. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. In *Proceedings of the ICLR*.
- [34] NVIDIA Developer. 2024. Jetson Modules, Support, Ecosystem, and Lineup. Retrieved July 16, 2024 from <https://developer.nvidia.com/embedded/jetson-modules>
- [35] Chenhe Dong, Yinghui Li, Haifan Gong, Miaoxin Chen, Junxin Li, Ying Shen, and Min Yang. 2022. A survey of natural language generation. *ACM Computing Surveys* 55, 8 (2022), 1–38.
- [36] Li Dong, Nan Yang, Wenhui Wang, Furu Wei, Xiaodong Liu, Yu Wang, Jianfeng Gao, Ming Zhou, and Hsiao-Wuen Hon. 2019. Unified language model pre-training for natural language understanding and generation. In *Proceedings of the NeurIPS*.

- [37] Jun Du, Tianyi Lin, Chunxiao Jiang, Qianqian Yang, C Faouzi Bader, and Zhu Han. 2024. Distributed foundation models for multi-modal learning in 6G wireless networks. *IEEE Wireless Communications* 31, 3 (2024), 20–30.
- [38] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. Retrieved from <https://arxiv.org/abs/2407.21783>
- [39] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. Retrieved from <https://arxiv.org/abs/2404.16130>
- [40] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme compression of large language models via additive quantization. In *Proceedings of the ICML*.
- [41] Samsung Exyno. 2024. Exynos 2400 with 10-core CPU Gets Detailed After Galaxy S24 Launch. Retrieved December 11, 2024 from <https://www.sammobile.com/news/exynos-2400-10-core-cpu-amd-rdna3-xclipse-940-gpu-specs-detailed>
- [42] Zichen Fan, Qirui Zhang, Pierre Abillama, Sara Shoori, Changwoo Lee, David Blaauw, Hun-Seok Kim, and Dennis Sylvester. 2023. TaskFusion: An efficient transfer learning architecture with dual delta sparsity for multi-task natural language processing. In *Proceedings of the ACM/IEEE ISCA*.
- [43] Elias Frantar and Dan Alistarh. 2022. Optimal brain compression: A framework for accurate post-training quantization and pruning. In *Proceedings of the NeurIPS*.
- [44] Elias Frantar and Dan Alistarh. 2023. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proceedings of the ICML*.
- [45] Elias Frantar, Saleh Ashkboos, Torsten Hoeftler, and Dan Alistarh. 2022. OPTQ: Accurate quantization for generative pre-trained transformers. In *Proceedings of the ICLR*.
- [46] NVIDIA GeForce. 2023. Compare Gaming Laptops: GeForce RTX 40 Series. Retrieved December 06, 2024 from <https://www.nvidia.com/en-us/geforce/laptops/compare/>
- [47] NVIDIA GeForce. 2023. Compare GeForce Graphics Cards. Retrieved December 06, 2024 from <https://www.nvidia.com/en-us/geforce/graphics-cards/compare/>
- [48] Zhengyang Geng, Meng-Hao Guo, Hongxu Chen, Xia Li, Ke Wei, and Zhouchen Lin. 2021. Is attention better than matrix decomposition? In *Proceedings of the ICLR*.
- [49] Georgi Gerganov. 2023. llama.cpp. Retrieved June 08, 2024 from <https://github.com/ggerganov/llama.cpp>
- [50] Google. 2024. Google Tensor: The Brains Behind Pixel Phones. Retrieved December 06, 2024 from https://store.google.com/intl/en_in/ideas/articles/google-tensor-pixel-smartphone/
- [51] Saurabh Goyal, Anamitra Roy Choudhury, Saurabh Raje, Venkatesan Chakaravarthy, Yogish Sabharwal, and Ashish Verma. 2020. Power-bert: Accelerating bert inference via progressive word-vector elimination. In *Proceedings of the ICML*.
- [52] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. Retrieved from <https://arxiv.org/abs/2312.00752>
- [53] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2023. MiniLLM: Knowledge distillation of large language models. In *Proceedings of the ICLR*.
- [54] Zhaopeng Gu, Bingke Zhu, Guibo Zhu, Yingying Chen, Ming Tang, and Jinqiao Wang. 2024. Anomalygpt: Detecting industrial anomalies using large vision-language models. In *Proceedings of the AAAI*.
- [55] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. 2023. Textbooks are all you need. Retrieved from <https://arxiv.org/abs/2306.11644>
- [56] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2023. OliVe: Accelerating large language models via hardware-friendly outlier-victim pair quantization. In *Proceedings of the ACM/IEEE ISCA*.
- [57] Cong Guo, Chen Zhang, Jingwen Leng, Zihan Liu, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2022. Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization. In *Proceedings of the IEEE/ACM MICRO*.
- [58] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. Retrieved from <https://arxiv.org/abs/2501.12948>
- [59] Liwei Guo, Wonkyo Choe, and Felix Xiaozhu Lin. 2023. STI: Turbocharge NLP inference at the edge via elastic pipelining. In *Proceedings of the ACM ASPLOS*.
- [60] Xiaotian Guo, Quan Jiang, Yixian Shen, Andy D. Pimentel, and Todor Stefanov. 2024. EASTER: Learning to split transformers at the edge robustly. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 11 (2024), 3626–3637.

- [61] Izzeddin Gur, Hiroki Furuta, Austin V. Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2024. A real-world WebAgent with planning, long context understanding, and program synthesis. In *Proceedings of the ICLR*.
- [62] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. 2023. MLX: Efficient and Flexible Machine Learning on Apple Silicon. Retrieved June 08, 2024 from <https://github.com/ml-explore>
- [63] Ying He, Jingcheng Fang, F. Richard Yu, and Victor C. Leung. 2024. Large language models (LLMs) inference offloading and resource allocation in cloud-edge computing: An active inference approach. *IEEE Transactions on Mobile Computing* 23, 12 (2024), 11253–11264.
- [64] Michael A. Hedderich, Lukas Lange, Heike Adel, Jannik Strötgen, and Dietrich Klakow. 2021. A survey on recent approaches for natural language processing in low-resource scenarios. In *Proceedings of the NAACL*.
- [65] Namgyu Ho, Laura Schmid, and Se-Young Yun. 2023. Large language models are reasoning teachers. In *Proceedings of the ACL*.
- [66] Yining Hong, Haoyu Zhen, Peihao Chen, Shuhong Zheng, Yilun Du, Zhenfang Chen, and Chuang Gan. 2023. 3d-llm: Injecting the 3d world into large language models. In *Proceedings of the NeurIPS*.
- [67] Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. 2024. A deep dive into large language models for automated bug localization and repair. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1471–1493.
- [68] Cheng-Yu Hsieh, Chun-Liang Li, Chih-kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alex Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes. In *Findings of ACL*.
- [69] Yen-Chang Hsu, Ting Hua, Sungen Chang, Qian Lou, Yilin Shen, and Hongxia Jin. 2022. Language model compression with weighted low-rank factorization. In *Proceedings of the ICLR*.
- [70] Chenghao Hu and Baochun Li. 2024. When the edge meets transformers: Distributed inference with transformer models. In *Proceedings of the IEEE ICDCS*.
- [71] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-rank adaptation of large language models. In *Proceedings of the ICLR*.
- [72] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. 2023. Inner monologue: Embodied reasoning through planning with language models. In *Proceedings of the CoRL*.
- [73] Intel. 2020. Intel Neural Compressor. Retrieved December 27, 2024 from <https://github.com/intel/neural-compressor>
- [74] Intel. 2022. Intel Launches 13th Gen Intel Core Processor Family Alongside New Intel Unison Solution. Retrieved December 11, 2024 from <https://www.intel.com/content/www/us/en/newsroom/news/13th-gen-core-launch.html>
- [75] Samvit Jain, Xun Zhang, Yuhao Zhou, Ganesh Ananthanarayanan, Junchen Jiang, Yuanchao Shu, Paramvir Bahl, and Joseph Gonzalez. 2020. Spatula: Efficient cross-camera video analytics on large camera networks. In *Proceedings of the IEEE/ACM SEC*.
- [76] Jongheon Jeong, Yang Zou, Taewan Kim, Dongqing Zhang, Avinash Ravichandran, and Onkar Dabeer. 2023. Winclip: Zero-/few-shot anomaly classification and segmentation. In *Proceedings of the IEEE/CVF CVPR*.
- [77] Tianchu Ji, Shraddhan Jain, Michael Ferdman, Peter Milder, H. Andrew Schwartz, and Niranjan Balasubramanian. 2021. On the distribution, sparsity, and inference-time quantization of attention values in transformers. In *Proceedings of the ACL-IJCNLP*.
- [78] Yixin Ji, Yang Xiang, Juntao Li, Wei Chen, Zhongyi Liu, Kehai Chen, and Min Zhang. 2024. Feature-based low-rank compression of large language models via bayesian optimization. In *Findings of EMNLP*.
- [79] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the EMNLP*.
- [80] Junchen Jiang, Yuhao Zhou, Ganesh Ananthanarayanan, Yuanchao Shu, and Andrew A. Chien. 2019. Networked cameras are the new big data clusters. In *Proceedings of the ACM Workshop on Hot Topics in Video Analytics and Intelligent Edges*.
- [81] Shiqi Jiang, Zhiqi Lin, Yuanchun Li, Yuanchao Shu, and Yunxin Liu. 2021. Flexible high-resolution object detection on edge devices with tunable latency. In *Proceedings of the ACM MobiCom*.
- [82] Yuxin Jiang, Chunkit Chan, Mingyang Chen, and Wei Wang. 2023. Lion: Adversarial distillation of proprietary large language models. In *Proceedings of the EMNLP*.
- [83] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. TinyBERT: Distilling BERT for natural language understanding. In *Proceedings of the EMNLP*.
- [84] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A quantitative and qualitative evaluation of LLM-based explainable fault localization. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1424–1446.
- [85] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. Retrieved from

<https://arxiv.org/abs/2001.08361>

- [86] Mehrdad Khani, Ganesh Ananthanarayanan, Kevin Hsieh, Junchen Jiang, Ravi Netravali, Yuanhao Shu, Mohammad Alizadeh, and Victor Bahl. 2023. RECL: Responsive resource-efficient continuous learning for video analytics. In *Proceedings of the USENIX NSDI*.
- [87] Geunwoo Kim, Pierre Baldi, and Stephen McAleer. 2024. Language models can solve computer tasks. In *Proceedings of the NeurIPS*.
- [88] Gyuwan Kim and Kyunghyun Cho. 2021. Length-adaptive transformer: Train once with length drop, use anytime with search. In *Proceedings of the ACL-IJCNLP*.
- [89] Minsoo Kim, Sihwa Lee, Janghwan Lee, Sukjin Hong, Du-Seong Chang, Wonyong Sung, and Jungwook Choi. 2023. Token-scaled logit distillation for ternary weight generative language models. In *Proceedings of the NeurIPS*.
- [90] Sangyeob Kim, Sangjin Kim, Wooyoung Jo, Soyeon Kim, Seongyon Hong, and Hoi-Jun Yoo. 2024. 20.5 C-Transformer: A 2.6-18.1 μ J/Token homogeneous DNN-Transformer/spiking-transformer processor with big-little network and implicit weight generation for large language models. In *Proceedings of the IEEE ISSCC*.
- [91] Sehoon Kim, Sheng Shen, David Thorsley, Amir Gholami, Woosuk Kwon, Joseph Hassoun, and Kurt Keutzer. 2022. Learned token pruning for transformers. In *Proceedings of the ACM SIGKDD*.
- [92] Huan Yee Koh, Jiaxin Ju, Ming Liu, and Shirui Pan. 2022. An empirical survey on long document summarization: Datasets, models, and metrics. *ACM Computing Surveys* (2022).
- [93] Jun Kong, Jin Wang, Liang-Chih Yu, and Xuejie Zhang. 2022. Accelerating inference for pretrained language models by unified multi-perspective early exiting. In *Proceedings of the COLING*.
- [94] Eldar Kurtic, Daniel Campos, Tuan Nguyen, Elias Frantar, Mark Kurtz, Benjamin Fineran, Michael Goin, and Dan Alistarh. 2022. The optimal BERT surgeon: Scalable and accurate second-order pruning for large language models. In *Proceedings of the EMNLP*.
- [95] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SOSP*.
- [96] Yanis Labrak, Adrien Bazoge, Emmanuel Morin, Pierre-Antoine Gourraud, Mickael Rouvier, and Richard Dufour. 2024. Biomistral: A collection of open-source pretrained large language models for medical domains. In *Findings of ACL*.
- [97] Huiyuan Lai and Malvina Nissim. 2024. A survey on automatic generation of figurative language: From rule-based systems to large language models. *ACM Computing Surveys* 56, 10 (2024), 1–34.
- [98] Varsha S. Lalapura, J. Amudha, and Hariram Selvamuruga Satheesh. 2022. Recurrent neural networks for edge intelligence: A survey. *ACM Computing Surveys* 54, 4, (2022), 1–38.
- [99] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. ALBERT: A lite BERT for self-supervised learning of language representations. In *Proceedings of the ICLR*.
- [100] Stefanos Laskaridis, Kleomenis Katevas, Lorenzo Minto, and Hamed Haddadi. 2024. MELting point: Mobile evaluation of language transformers. In *Proceedings of the ACM MobiCom*.
- [101] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. 2024. OWQ: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models. In *Proceedings of the AAAI*.
- [102] Juhyeon Lee, Insung Bahk, Hoseung Kim, Sinjin Jeong, Suyeon Lee, and Donghyun Min. 2024. An autonomous parallelization of transformer model inference on heterogeneous edge devices. In *Proceedings of the ACM ICS*.
- [103] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the ICML*.
- [104] Bingbing Li, Zhenglun Kong, Tianyun Zhang, Ji Li, Zhengang Li, Hang Liu, and Caiwen Ding. 2020. Efficient transformer-based large scale language representations using hardware-friendly block structured pruning. In *Proceedings of the EMNLP*.
- [105] Hongxin Li, Jingran Su, Yuntao Chen, Qing Li, and ZHAO-XIANG ZHANG. 2024. SheetCopilot: Bringing software productivity to the next level through large language models. In *Proceedings of the NeurIPS*.
- [106] Lunian Harold Li, Jack Hessel, Youngjae Yu, Xiang Ren, Kai-Wei Chang, and Yejin Choi. 2023. Symbolic chain-of-thought distillation: Small models can also “Think” step-by-step. In *Proceedings of the ACL*.
- [107] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. 2023. Textbooks are all you need ii: Phi-1.5 technical report. Retrieved from <https://arxiv.org/abs/2309.05463>
- [108] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. 2024. Personal llm agents: Insights and survey about the capability, efficiency and security. Retrieved from <https://arxiv.org/abs/2401.05459>
- [109] Yixiao Li, Yifan Yu, Qingru Zhang, Chen Liang, Pengcheng He, Weizhu Chen, and Tuo Zhao. 2023. LoSparse: Structured compression of large language models based on low-rank and sparse approximation. In *Proceedings of the ICML*.

- [110] Chen Liang, Simiao Zuo, Qingru Zhang, Pengcheng He, Weizhu Chen, and Tuo Zhao. 2023. Less is more: Task-aware layer-wise distillation for language model compression. In *Proceedings of the ICML*.
- [111] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirum, Yonatan Belinkov, Shai Shalev-Shwartz, et al. 2024. Jamba: A hybrid transformer-mamba language model. Retrieved from <https://arxiv.org/abs/2403.19887>
- [112] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of the MLSys*.
- [113] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. Retrieved from <https://arxiv.org/abs/2412.19437>
- [114] Jing Liu, Ruihao Gong, Xiuying Wei, Zhiwei Dong, Jianfei Cai, and Bohan Zhuang. 2023. QLLM: Accurate and efficient low-bitwidth quantization for large language models. In *Proceedings of the ICLR*.
- [115] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys* 55, 9 (2023), 1–35.
- [116] Weijie Liu, Peng Zhou, Zhiruo Wang, Zhe Zhao, Haotang Deng, and Qi Ju. 2020. FastBERT: A self-distilling BERT with adaptive inference time. In *Proceedings of the ACL*.
- [117] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, et al. 2024. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. In *Proceedings of the ICML*.
- [118] Liqiang Lu, Yicheng Jin, Hangrui Bi, Zizhang Luo, Peng Li, Tao Wang, and Yun Liang. 2021. Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture. In *Proceedings of the IEEE/ACM MICRO*.
- [119] Ming Y. Lu, Bowen Chen, Drew F. K. Williamson, Richard J. Chen, Melissa Zhao, Aaron K. Chow, Kenji Ikemura, Ahnong Kim, Dimitra Pouli, Ankush Patel, et al. 2024. A multimodal generative AI copilot for human pathology. *Nature* 634, 8033 (2024), 466–473.
- [120] Yan Lu, Shiqi Jiang, Ting Cao, and Yuanchao Shu. 2022. Turbo: Opportunistic enhancement for edge video analytics. In *Proceedings of the ACM SenSys*.
- [121] Yan Lu, Yuanchao Shu, Xu Tan, Yunxin Liu, Mengyu Zhou, Qi Chen, and Dan Pei. 2019. Collaborative learning between cloud and end devices: An empirical study on location prediction. In *Proceedings of the ACM/IEEE SEC*.
- [122] Yan Lu, Zhun Zhong, and Yuanchao Shu. 2023. Multi-view domain adaptive object detection on camera networks. In *Proceedings of the AAAI*.
- [123] Ruilong Ma, Xiang Yang, Jingyu Wang, Qi Qi, Haifeng Sun, Jing Wang, Zirui Zhuang, and Jianxin Liao. 2024. HPipe: Large language model pipeline parallelism for long context on heterogeneous cost-effective devices. In *Proceedings of the NAACL-HLT*.
- [124] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. LLM-Pruner: On the structural pruning of large language models. In *Proceedings of the NeurIPS*.
- [125] Xinbei Ma, Zhuosheng Zhang, and Hai Zhao. 2024. Coco-agent: A comprehensive cognitive mllm agent for smart-phone gui automation. In *Findings of ACL*.
- [126] Sachin Mehta, Marjan Ghazvininejad, Srinivasan Iyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2021. Delight: Deep and light-weight transformer. In *Proceedings of the ICLR*.
- [127] Sachin Mehta, Mohammad Hossein Sekhavat, Qingqing Cao, Maxwell Horton, Yanzi Jin, Chenfan Sun, Iman Mirzadeh, Mahyar Najibi, Dmitry Belenko, Peter Zatloukal, et al. 2024. OpenELM: An efficient language model family with open-source training and inference framework. Retrieved from <https://arxiv.org/abs/2404.14619>
- [128] Microsoft. 2018. ONNX Runtime is a Cross-platform Inference and Training Machine-learning Accelerator. Retrieved December 29, 2024 from <https://github.com/microsoft/onnxruntime>
- [129] Microsoft. 2023. Microsoft Copilot. Retrieved June 14, 2024 from <https://copilot.microsoft.com/>
- [130] Sébastien Bubeck Moján Javaheripi. 2023. Phi-2: The Surprising Power of Small Language Models. Retrieved June 01, 2024 from <https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/>
- [131] Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. 2021. DNNFusion: Accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the ACM PLDI*.
- [132] Wei Niu, Md Musfiqur Rahman Sanim, Zhihao Shu, Jiexiong Guan, Xipeng Shen, Miao Yin, Gagan Agrawal, and Bin Ren. 2024. SmartMem: Layout transformation elimination and adaptation for efficient DNN execution on mobile. In *Proceedings of the ACM ASPLOS*.
- [133] NVIDIA. 2023. TensorRT-LLM. Retrieved June 09, 2024 from <https://github.com/NVIDIA/TensorRT-LLM>
- [134] NVIDIA. 2024. Minitron-4B-Base. Retrieved December 19, 2024 from <https://huggingface.co/nvidia/Minitron-4B-Base>
- [135] OpenAI. 2022. Introducing ChatGPT. Retrieved July 05, 2024 from <https://openai.com/index/chatgpt/>

- [136] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *Proceedings of the NeurIPS*.
- [137] Arthi Padmanabhan, Neil Agarwal, Anand Iyer, Ganesh Ananthanarayanan, Yuanchao Shu, Nikolaos Karianakis, Guoqing Harry Xu, and Ravi Netravali. 2023. Gemel: Model merging for memory-efficient, real-time video analytics at the edge. In *Proceedings of the USENIX NSDI*.
- [138] Shankar Padmanabhan, Yasumasa Onoe, Michael Zhang, Greg Durrett, and Eunsol Choi. 2024. Propagating knowledge updates to lms through distillation. In *Proceedings of the NeurIPS*.
- [139] Chenbin Pan, Burhaneddin Yaman, Tommaso Nesti, Abhirup Mallik, Alessandro G Allievi, Senem Velipasalar, and Liu Ren. 2024. VLP: Vision language planning for autonomous driving. In *Proceedings of the IEEE/CVF CVPR*.
- [140] Jupinder Parmar, Shrimai Prabhumoye, Joseph Jennings, Mostofa Patwary, Sandeep Subramanian, Dan Su, Chen Zhu, Deepak Narayanan, Aastha Jhunjhunwala, Ayush Dattagupta, et al. 2024. Nemotron-4 15B technical report. Retrieved from <https://arxiv.org/abs/2402.16819>
- [141] Raspberry Pi. 2019. Raspberry Pi 4 on Sale Now. Retrieved December 29, 2024 from <https://www.raspberrypi.com/news/raspberry-pi-4-on-sale-now-from-35/>
- [142] Pytorch. 2023. ExecuTorch. Retrieved December 12, 2024 from <https://github.com/pytorch/executorch>
- [143] Biqing Qi, Xinquan Chen, Junqi Gao, Dong Li, Jianxing Liu, Ligang Wu, and Bowen Zhou. 2024. Interactive continual learning: Fast and slow thinking. In *Proceedings of the IEEE/CVF CVPR*.
- [144] Guanjiao Qu, Qiyuan Chen, Wei Wei, Zheng Lin, Xianhao Chen, and Kaibin Huang. 2024. Mobile edge intelligence for large language models: A contemporary survey. Retrieved from <https://arxiv.org/abs/2407.18921>
- [145] Qualcomm. 2024. Snapdragon 8 Series Mobile Platforms | Qualcomm. Retrieved July 17, 2024 from <https://www.qualcomm.com/products/mobile/snapdragon/smartphones/snapdragon-8-series-mobile-platforms>
- [146] Brian Rakowski. 2023. Pixel 8 Pro – the First Smartphone with AI Built in – is Now Running Gemini Nano, Plus More AI Updates Coming to the Pixel Portfolio. Retrieved December 09, 2024 from <https://blog.google/products/pixel/pixel-feature-drop-december-2023/>
- [147] Reza Rawassizadeh and Yi Rong. 2023. ODSearcH: Fast and resource efficient on-device natural language search for fitness trackers' data. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 6, 4 (2023), 1–25.
- [148] Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Observational scaling laws and the predictability of language model performance. Retrieved from <https://arxiv.org/abs/2405.10938>
- [149] Rajarshi Saha, Varun Srivastava, and Mert Pilanci. 2023. Matrix compression via randomized low rank and low precision factorization. In *Proceedings of the NeurIPS*.
- [150] Victor Sanh, Thomas Wolf, and Alexander Rush. 2020. Movement pruning: Adaptive sparsity by fine-tuning. In *Proceedings of the NeurIPS*.
- [151] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2023. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *Proceedings of the ICLR*.
- [152] Sheng Shen, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. 2020. Q-BERT: Hessian based ultra low precision quantization of BERT. In *Proceedings of the AAAI*.
- [153] Xuan Shen, Peiyan Dong, Lei Lu, Zhenglun Kong, Zhengang Li, Ming Lin, Chao Wu, and Yanzhi Wang. 2024. Agile-quant: Activation-guided quantization for faster inference of LLMs on the edge. In *Proceedings of the AAAI*.
- [154] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the ICML*.
- [155] Kumar Shridhar, Alessandro Stolfo, and Mrinmaya Sachan. 2023. Distilling reasoning capabilities into smaller language models. In *Proceedings of the ACL*.
- [156] Md Maruf Hossain Shuvo, Syed Kamrul Islam, Jianlin Cheng, and Bashir I. Morshed. 2022. Efficient acceleration of deep learning inference on resource-constrained edge devices: A review. *Proceedings of the IEEE* 111, 1 (2022), 42–91.
- [157] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SOSP*.
- [158] Shrihari Sridharan, Jacob R. Stevens, Kaushik Roy, and Anand Raghunathan. 2023. X-former: In-memory acceleration of transformers. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 31, 8 (2023), 1223–1233.
- [159] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2024. RoFormer: Enhanced transformer with rotary position embedding. *Elsevier Neurocomputing* 568, 1 (2024), 1–14.
- [160] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2024. A simple and effective pruning approach for large language models. In *Proceedings of the ICLR*.

- [161] Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. 2024. Spectr: Fast speculative decoding via optimal transport. In *Proceedings of the NeurIPS*.
- [162] Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. 2020. MobileBERT: A compact task-agnostic BERT for resource-limited devices. In *Proceedings of the ACL*.
- [163] Thierry Tambe, Coleman Hooper, Lillian Pentecost, Tianyu Jia, En-Yu Yang, Marco Donato, Victor Sanh, Paul Whatmough, Alexander M. Rush, David Brooks, and Gu-Yeon Wei. 2021. EdgeBERT: Sentence-level energy optimizations for latency-aware multi-task NLP inference. In *Proceedings of the IEEE/ACM MICRO*.
- [164] Thierry Tambe, En-Yu Yang, Zishen Wan, Yuntian Deng, Vijay Janapa Reddi, Alexander Rush, David Brooks, and Gu-Yeon Wei. 2020. Algorithm-hardware co-design of adaptive floating-point encodings for resilient deep learning inference. In *Proceedings of the ACM/IEEE DAC*.
- [165] Thierry Tambe, Jeff Zhang, Coleman Hooper, Tianyu Jia, Paul N. Whatmough, Joseph Zuckerman, Maico Cassel Dos Santos, Erik Jens Loscalzo, Davide Giri, Kenneth Shepard, et al. 2023. 22.9 A 12nm 18.1 TFLOPs/W sparse transformer processor with entropy-based early exit, mixed-precision predication and fine-grained power management. In *Proceedings of the IEEE ISSCC*.
- [166] Thierry Tambe, Jeff Zhang, Coleman Hooper, Tianyu Jia, Paul N. Whatmough, Joseph Zuckerman, Maico Cassel Dos Santos, Erik Jens Loscalzo, Davide Giri, Kenneth Shepard, et al. 2023. 22.9 A 12nm 18.1TFLOPs/W sparse transformer processor with entropy-based early exit, mixed-precision predication and fine-grained power management. In *Proceedings of the IEEE ISSCC*.
- [167] Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Lixin Su, Suqi Cheng, Dawei Yin, and Chao Huang. 2024. Graphgpt: Graph instruction tuning for large language models. In *Proceedings of the ACM SIGIR*. 491–500.
- [168] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: A family of highly capable multimodal models. Retrieved from <https://arxiv.org/abs/2312.11805>
- [169] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. Retrieved from <https://arxiv.org/abs/2403.08295>
- [170] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving open language models at a practical size. Retrieved from <https://arxiv.org/abs/2408.00118>
- [171] MLC Team. 2023. MLC LLM. Retrieved June 06, 2024 from <https://github.com/mlc-ai/mlc-llm>
- [172] Qualcomm Team. 2024. Snapdragon 8 Gen 3 Mobile Platform. Retrieved December 03, 2024 from <https://www.qualcomm.com/products/mobile/snapdragon/smartphones/snapdragon-8-series-mobile-platforms/snapdragon-8-gen-3>
- [173] Wenbin Tian, Chaojie Gu, Miao Guo, Shibo He, Jiawen Kang, Dusit Niyato, and Jiming Chen. 2024. Large-scale deterministic networks: Architecture, enabling technologies, case study and future directions. In *Proceedings of the IEEE Network*.
- [174] Xiaoyu Tian, Junru Gu, Bailin Li, Yicheng Liu, Yang Wang, Zhiyong Zhao, Kun Zhan, Peng Jia, Xianpeng Lang, and Hang Zhao. 2024. Drivevlm: The convergence of autonomous driving and large vision-language models. Retrieved from <https://arxiv.org/abs/2402.12289>
- [175] Yuanhe Tian, Fei Xia, and Yan Song. 2024. Dialogue summarization with mixture of experts based on large language models. In *Proceedings of the ACL*.
- [176] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. Retrieved from <https://arxiv.org/abs/2302.13971>
- [177] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. Retrieved from <https://arxiv.org/abs/2307.09288>
- [178] Marcos Treviso, Ji-Ung Lee, Tianchu Ji, Betty van Aken, Qingqing Cao, Manuel R. Ciosici, Michael Hassid, Kenneth Heafield, Sara Hooker, Colin Raffel, et al. 2023. Efficient methods for natural language processing: A survey. *Transactions of the Association for Computational Linguistics* 1, 11 (2023), 826–860.
- [179] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. 2024. Quip#: Even better LLM quantization with hadamard incoherence and lattice codebooks. In *Proceedings of the ICML*.
- [180] Fengbin Tu, Zihan Wu, Yiqi Wang, Ling Liang, Liu Liu, Yufei Ding, Leibo Liu, Shaojun Wei, Yuan Xie, and Shouyi Yin. 2022. A 28nm 15.59 μ J/token full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline-/parallel reconfigurable modes. In *Proceedings of the IEEE ISSCC*.
- [181] Fengbin Tu, Zihan Wu, Yiqi Wang, Weiwei Wu, Leibo Liu, Yang Hu, Shaojun Wei, and Shouyi Yin. 2023. 16.1 MuTCIM: A 28nm 2.24 μ J/Token attention-token-bit hybrid sparse digital CIM-Based accelerator for multimodal transformers. In *Proceedings of the IEEE ISSCC*.

- [182] Shikhar Tuli and Niraj K. Jha. 2023. AccelTran: A sparsity-aware accelerator for dynamic inference with transformers. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 11 (2023), 4038–4051.
- [183] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the NeurIPS*.
- [184] Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. 2024. Knowledge fusion of large language models. In *Proceedings of the ICLR*.
- [185] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. 2024. Efficient large language models: A survey. *Transactions on Machine Learning Research* 1, 1 (2024), 1–67.
- [186] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. SpAtten: Efficient sparse attention architecture with cascade token and head pruning. In *Proceedings of the IEEE HPCA*.
- [187] Rongkai Wang, Yiyang Jing, Chaojie Gu, Shibo He, and Jiming Chen. 2025. End-to-end multitarget flexible job shop scheduling with deep reinforcement learning. *IEEE Internet of Things Journal* 12, 4 (2025), 4420–4434.
- [188] Wenhui Wang, Hangbo Bao, Shaohan Huang, Li Dong, and Furu Wei. 2021. MiniLMv2: Multi-head self-attention relation distillation for compressing pretrained transformers. In *Proceedings of the ACL-IJCNLP*.
- [189] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Proceedings of the NeurIPS*.
- [190] Xiaofei Wang, Yiwen Han, Victor C. M. Leung, Dusit Niyato, Xueqiang Yan, and Xu Chen. 2020. Convergence of edge computing and deep learning: A comprehensive survey. *IEEE Communications Surveys and Tutorials* 22, 2 (2020), 869–904.
- [191] Yiding Wang, Kai Chen, Haisheng Tan, and Kun Guo. 2023. Tabi: An efficient multi-level inference system for large language models. In *Proceedings of the ACM EuroSys*.
- [192] Yingchao Wang, Chen Yang, Shulin Lan, Liehuang Zhu, and Yan Zhang. 2024. End-edge-cloud collaborative computing for deep learning: A comprehensive survey. *IEEE Communications Surveys and Tutorials* 26, 4 (2024), 2647–2683.
- [193] Zichen Wang, Feng Yan, Tianyi Wang, Cong Wang, Yuanchao Shu, Peng Cheng, and Jiming Chen. 2025. Fed-DFA: Federated distillation for heterogeneous model fusion through the adversarial lens. In *Proceedings of the AAAI*.
- [194] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. AutoDroid: LLM-powered task automation in Android. In *Proceedings of the ACM MobiCom*.
- [195] Minghao Wu, Abdul Waheed, Chiyu Zhang, Muhammad Abdul-Mageed, and Alham Aji. 2024. LaMini-LM: A diverse herd of distilled models from large-scale instructions. In *Proceedings of the EACL*.
- [196] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. 2024. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *Proceedings of the ICML*.
- [197] Mengzhou Xia, Zexuan Zhong, and Danqi Chen. 2022. Structured pruning learns compact and accurate models. In *Proceedings of the ACL*.
- [198] Jiannan Xiang, Tianhua Tao, Yi Gu, Tianmin Shu, Zirui Wang, Zichao Yang, and Zhiting Hu. 2024. Language models meet world models: Embodied experiences enhance language models. In *Proceedings of the NeurIPS*.
- [199] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the ICML*.
- [200] Canwen Xu and Julian McAuley. 2023. A survey on model compression and acceleration for pretrained language models. In *Proceedings of the AAAI*.
- [201] Daliang Xu, Wangsong Yin, Hao Zhang, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2024. EdgeLLM: Fast On-device LLM inference with speculative decoding. *IEEE Transactions on Mobile Computing* 1, 1 (2024), 1–18.
- [202] Lilin Xu, Chaojie Gu, Rui Tan, Shibo He, and Jiming Chen. 2023. MESEN: Exploit multimodal data to design unimodal human activity recognition with few labels. In *Proceedings of the ACM SenSys*.
- [203] Lilin Xu, Keyi Wang, Chaojie Gu, Xiuzhen Guo, Shibo He, and Jiming Chen. 2024. GesturePrint: Enabling user identification for mmWave-based gesture recognition systems. In *Proceedings of the IEEE ICDCS*.
- [204] Peng Xu, Wenqi Shao, Mengzhao Chen, Shitao Tang, Kaipeng Zhang, Peng Gao, Fengwei An, Yu Qiao, and Ping Luo. 2024. BESA: Pruning large language models with blockwise parameter-efficient sparsity allocation. In *Proceedings of the ICLR*.
- [205] Xuhai Xu, Bingsheng Yao, Yuanzhe Dong, Saadia Gabriel, Hong Yu, James Hendler, Marzyeh Ghassemi, Anind K. Dey, and Dakuo Wang. 2024. Mental-llm: Leveraging large language models for mental health prediction via online text data. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 8, 1 (2024), 1–32.
- [206] Yuxuan Yan, Qianqian Yang, Shunpu Tang, and Zhiguo Shi. 2024. Federa: Efficient fine-tuning of language models in federated learning leveraging weight decomposition. Retrieved from <https://arxiv.org/abs/2404.18848>
- [207] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical report. Retrieved from <https://arxiv.org/abs/2407.10671>

- [208] Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent Hellendoorn. 2024. Large language models for test-free fault localization. In *Proceedings of the ACM/IEEE ICSE*.
- [209] Kailai Yang, Tianlin Zhang, Ziyang Kuang, Qianqian Xie, Jimin Huang, and Sophia Ananiadou. 2024. MentaLLaMA: Interpretable mental health analysis on social media with large language models. In *Proceedings of the ACM WWW*.
- [210] Yongjie Yang, Tao Chen, Yujing Huang, Xiuzhen Guo, and Longfei Shangguan. 2024. MAF: Exploring mobile acoustic field for hand-to-face gesture interactions. In *Proceedings of the ACM CHI*.
- [211] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022. ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers. In *Proceedings of the NeurIPS*.
- [212] Chong Yu, Tao Chen, and Zhongxue Gan. 2023. Boost transformer-based language models with GPU-Friendly sparsity and quantization. In *Findings of ACL*.
- [213] Jinliang Yuan, Chen Yang, Dongqi Cai, Shihe Wang, Xin Yuan, Zeling Zhang, Xiang Li, Dingge Zhang, Hanzi Mei, Xianqing Jia, et al. 2024. Mobile foundation model as firmware. In *Proceedings of the ACM MobiCom*.
- [214] Ali Hadi Zadeh, Isak Edo, Omar Mohamed Awad, and Andreas Moshovos. 2020. Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference. In *Proceedings of the IEEE/ACM MICRO*.
- [215] Ali Hadi Zadeh, Mostafa Mahmoud, Ameer Abdelhadi, and Andreas Moshovos. 2022. Mokey: Enabling narrow fixed-point inference for out-of-the-box floating-point transformer models. In *Proceedings of the ACM/IEEE ISCA*.
- [216] Ziqian Zeng, Yihuai Hong, Hongliang Dai, Huiping Zhuang, and Cen Chen. 2024. ConsistentEE: A consistent and hardness-guided early exiting method for accelerating language models inference. In *Proceedings of the AAAI*.
- [217] Chen Zhang, Yang Yang, Jiahao Liu, Jingang Wang, Yunsen Xian, Benyou Wang, and Dawei Song. 2023. Lifting the curse of capacity gap in distilling language models. In *Proceedings of the ACL*.
- [218] Li Zhang, Shihe Wang, Xianqing Jia, Zhihan Zheng, Yunhe Yan, Longxi Gao, Yuanchun Li, and Mengwei Xu. 2024. LlamaTouch: A faithful and scalable testbed for mobile UI automation task evaluation. In *Proceedings of the ACM UIST*.
- [219] Mingyang Zhang, Hao Chen, Chunhua Shen, Zhen Yang, Linlin Ou, Xinyi Yu, and Bohan Zhuang. 2024. LoRAPrune: Structured pruning meets low-rank parameter-efficient fine-tuning. In *Findings of ACL*.
- [220] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. Retrieved from <https://arxiv.org/abs/2205.01068>
- [221] Wei Zhang, Lu Hou, Yichun Yin, Lifeng Shang, Xiao Chen, Xin Jiang, and Qun Liu. 2020. TernaryBERT: Distillation-aware Ultra-low Bit BERT. In *Proceedings of the EMNLP*.
- [222] Xiaojie Zhang and Saptarshi Debroy. 2023. Resource management in mobile edge computing: A comprehensive survey. *ACM Computing Surveys* 55, 131 (2023), 1–37.
- [223] Xinyuan Zhang, Jiangtian Nie, Yudong Huang, Gaochang Xie, Zehui Xiong, Jiang Liu, Dusit Niyato, and Xuemin Sherman Shen. 2024. Beyond the cloud: Edge inference for generative large language models in wireless networks. *IEEE Transactions on Wireless Communications* 24, 1 (2024), 643–658.
- [224] Yingtao Zhang, Haoli Bai, Haokun Lin, Jialin Zhao, Lu Hou, and Carlo Vittorio Cannistraci. 2024. Plug-and-play: An efficient post-training pruning method for large language models. In *Proceedings of the ICLR*.
- [225] Yiwen Zhang, Xumiao Zhang, Ganesh Ananthanarayanan, Anand Iyer, Yuanchao Shu, Victor Bahl, Z. Morley Mao, and Mosharaf Chowdhury. 2024. Vulcan: Automatic query planning for live ML analytics. In *Proceedings of the USENIX NSDI*.
- [226] Zhenyu Zhang, Shiwei Liu, Runjin Chen, Bhavya Kailkhura, Beidi Chen, and Atlas Wang. 2024. Q-Hitter: A better token oracle for efficient LLM inference via sparse-quantized KV cache. In *Proceedings of the MLSys*.
- [227] Junchen Zhao, Yurun Song, Simeng Liu, Ian Harris, and Sangeetha Abdu Jyothi. 2024. LinguaLinked: Distributed large language model inference on mobile devices. In *Proceedings of the ACL*.
- [228] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the USENIX OSDI*.
- [229] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. Transpim: A memory-based acceleration via software-hardware co-design for transformer. In *Proceedings of the IEEE HPCA*.
- [230] Qihang Zhou, Guansong Pang, Yu Tian, Shibo He, and Jiming Chen. 2024. AnomalyCLIP: Object-agnostic prompt learning for zero-shot anomaly detection. In *Proceedings of the ICLR*.
- [231] Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. 2020. BERT loses patience: Fast and robust inference with early exit. In *Proceedings of the NeurIPS*.
- [232] Jiaqi Zhu, Shaofeng Cai, Fang Deng, Beng Chin Ooi, and Junran Wu. 2024. Do LLMs understand visual anomalies? Uncovering LLM’s capabilities in zero-shot anomaly detection. In *Proceedings of the ACM MM*.
- [233] Wei Zhu. 2021. LeeBERT: Learned early exit for BERT with cross-level optimization. In *Proceedings of the ACL*.

- [234] Yun Zhu, Yinxiao Liu, Felix Stahlberg, Shankar Kumar, Yu-Hui Chen, Liangchen Luo, Lei Shu, Renjie Liu, Jindong Chen, and Lei Meng. 2024. Towards an on-device agent for text rewriting. In *Findings of NAACL*.

Received 10 September 2024; revised 9 January 2025; accepted 14 February 2025